DIGITAL SYSTEM DESIGN WITH VERILOG USING AI TOOLS

Chapter 10

Basic System on Chip

SoC Architecture, RV32I Microprocessor, Memory, and Address Bus



RISC-V RV32I Single-Cycle Processor · DE0 FPGA

VS Code + AI Assistant · Icarus Verilog · GTKWave / WaveTrace · RARS · Quartus II

Agenda

Today's Roadmap

From building blocks to a microprocessor, then from a microprocessor to a System on Chip — and finally building it yourself in the lab.

Theory 1 (0–45 min)	Theory 2 (45–80 min)	Lab (80–180 min)	Wrap-up
Sections 01–07	**Sections 08–11**	**Sections 12–14**	**Section 15**
ISA, RV32I formats, single-cycle datapath, control unit, next-PC, timing	Memory, bus and address decoding, memory-mapped I/O, FPGA reality	Build the RV32I core, bus and GPIO with AI assistance; verify and demo on DE0	The big picture, review questions and the lab mission

The one idea to carry through the whole chapter

For the CPU, everything is memory. Turning on an LED is a sw to an address; reading the switches is a lw from an address.

01

From Digital Logic to SoC

Why this matters

You already built the ALU, registers and memory. This chapter connects them into a processor that runs real programs — and then into a small System on Chip.

What Are We Building?

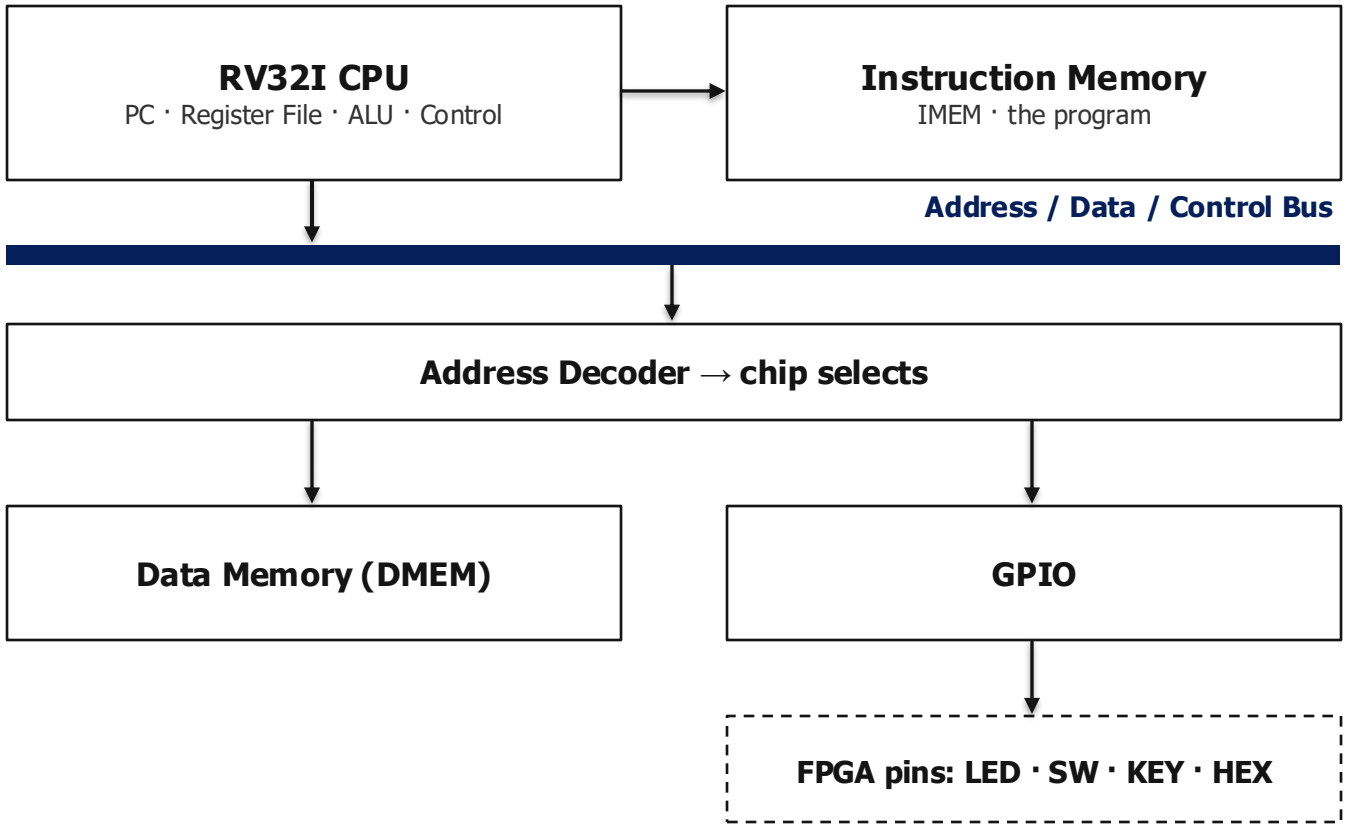
Goal: build a RISC-V microprocessor in Verilog and connect it to Memory and I/O through an Address Bus.

Components from earlier chapters

- ALU
- Register
- Register File
- Multiplexer
- Memory
- Control Logic

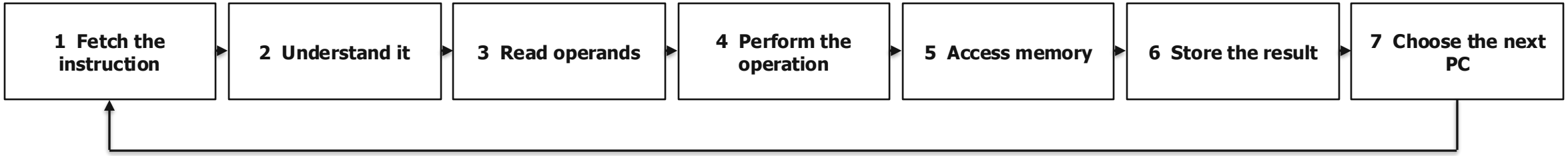
Now

We connect them into one system that can execute a real program.



The Big Question: Can We Build a Computer?

If we already have an ALU, registers and memory — a processor still needs more than computation. For every instruction it must:



…then repeat for the next instruction, forever

Step	Hardware that does it	Where you have seen it
Fetch / next PC	PC register + Instruction Memory + adders	Registers, memory (earlier chapters)
Understand	Control Unit (decoder)	Combinational logic, decoders
Read / store	Register File	Register file chapter
Operate / access memory	ALU, Data Memory	ALU and memory chapters

From Components to a System

A processor is two cooperating parts: a datapath that moves data, and control that decides what the datapath must do.

Datapath — moves and transforms data

PC address of the current instruction
Register File x0 … x31
ALU computes results and addresses
Memory instructions and data
Multiplexers choose between sources

Control — decides what the datapath must do

- Which register is written?
- Which ALU operation is selected?
- Is memory read or written?
- Where does the next PC come from?



Microprocessor, Microcontroller, System on Chip

All three share one principle: the CPU talks to everything through a bus, using addresses. What differs is how much is integrated on the chip.

System	Contains	Memory	Examples
Microprocessor	CPU core only	Outside the chip	Intel Core, AMD Ryzen
Microcontroller	CPU + Memory + basic peripherals	On chip (Flash, SRAM)	AVR, STM32, ESP32-C3
System on Chip (SoC)	CPU (maybe multi-core) + Memory + Peripherals + Accelerators + Interconnect	On chip and/or external DRAM	Apple M-series, Snapdragon, RK3588

The SoC we build in this chapter



RV32I CPU + Instruction Memory + Data Memory + Address Bus + Address Decoder + GPIO + external FPGA I/O = a small SoC

For the CPU, Everything Is Memory

The CPU does not need a special instruction for each device. The address decides whether it reaches RAM, an LED, the switches, or any other peripheral.

Instructions the CPU does NOT need

```
TURN_LED_ON   ✗
READ_SWITCH   ✗
WRITE_DISPLAY   ✗
```

What it uses instead

```
sw t0, 0(s0)    # write to an address
lw t0, 4(s0)    # read from an address
```

With s0 = 0x2000_0000, the first line lights the LEDs and the second reads the switches — ordinary loads and stores.

Keep in mind

We will build toward this idea throughout the chapter: datapath → bus → memory map → memory-mapped I/O.

Learning Outcomes

By the end of this chapter, students can:

#	Outcome	Where
1	Explain the RV32I ISA and its instruction formats	Sections 02–03
2	Explain a single-cycle datapath	Sections 04–05
3	Build an RV32I microprocessor using Verilog	Lab Parts A–C
4	Connect datapath and control into a CPU that runs real programs	Sections 05–07, Lab C
5	Design a Memory Map and an Address Decoder	Sections 09–10
6	Connect Memory and GPIO through an Address Bus	Lab Part D
7	Verify the processor with testbench and waveform	Section 14
8	Run the processor on the DE0 FPGA	Lab Part E
9	Use AI coding tools through Spec → Generate → Verify → Explain	Section 13

02

RISC-V and RV32I

Why this matters

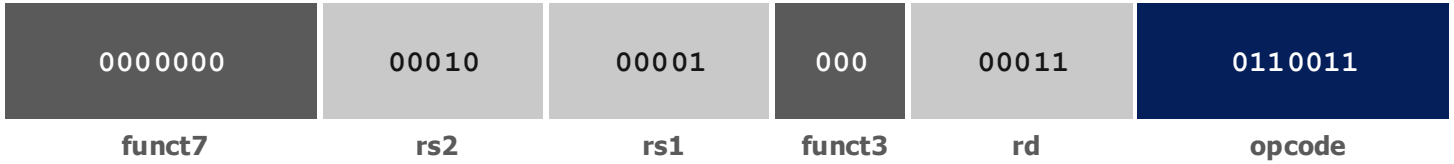
Before building hardware we need the contract it must honour: which instructions exist, which registers, and how each instruction is encoded in bits.

How Does Software Control Hardware?

Humans read “add the values in x1 and x2 and store the result in x3”. Hardware sees only 32 bits.

```
add x3, x1, x2
```

Software writes an instruction. The CPU must discover the operation, the registers and what to do next — from bits alone.



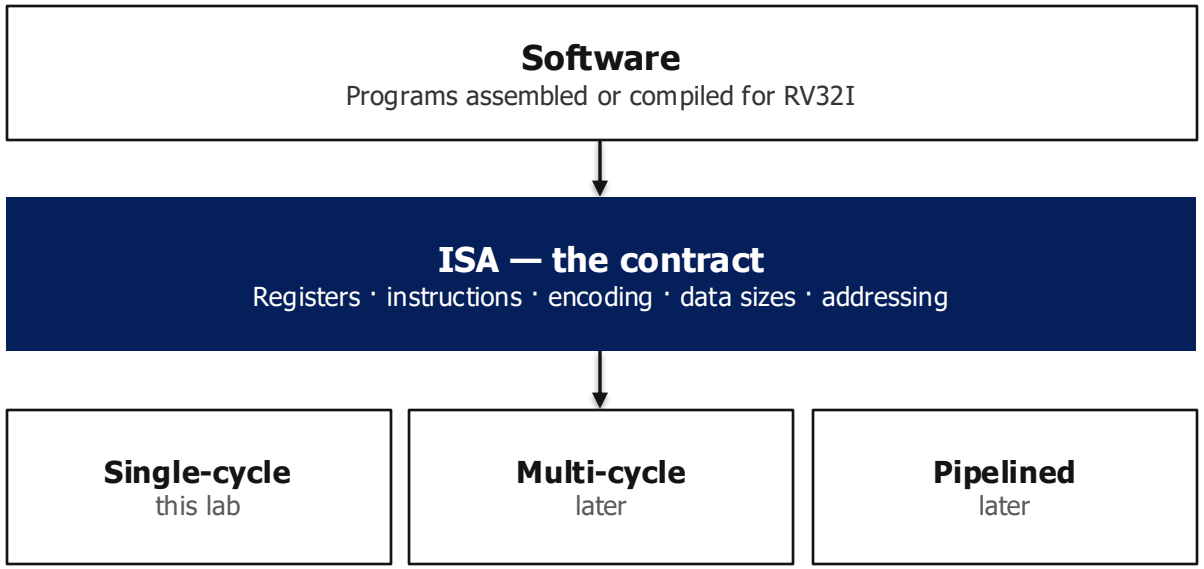
= `0x002081B3` — one 32-bit word in instruction memory

Question

How does software describe an operation in a form hardware can understand? → the Instruction Set Architecture (ISA).

ISA: The Software–Hardware Contract

Processors can be built very differently inside and still run the same software — if they implement the same ISA.



Different implementations — the same RV32I program runs on all of them

The ISA specifies

- Available instructions
- Registers
- Instruction encoding
- Data sizes
- Memory addressing behavior

Why RISC-V?

An open, simple and industrially relevant ISA — ideal for building a CPU yourself.

Open standard

Licensing No license fee — anyone may design a CPU that runs RISC-V code

Small, simple base

Size The RV32I base has about 40 instructions — small enough to build in a lab

Used in industry

Examples ESP32-C3, GD32VF103, SiFive cores, and microcontrollers inside many SoCs

We use RV32I

Part	Meaning
RV	RISC-V
32	32-bit architecture — registers, addresses and instructions
I	Base Integer Instruction Set

RV32I at a Glance

Six facts that shape every piece of hardware we build.

32-bit

Registers, addresses and instructions are all 32 bits wide.

x0 – x31

32 integer registers in the Register File.

x0 = 0

Always reads zero. Writing to x0 has no effect.

Load / Store

Compute only between registers; memory only through lw / sw.

Byte-addressed

1 address = 1 byte, stored little-endian (LSB at the lowest address).

PC + 4

Every instruction is 4 bytes, so normally PCNext = PC + 4.

Registers You Will See

Hardware sees only register numbers — 5-bit fields rs1, rs2, rd select one of 32. ABI names make assembly readable for humans.

Register	ABI name	Typical use
x0	zero	Constant 0
x1	ra	Return address
x2	sp	Stack pointer
x5–x7, x28–x31	t0–t6	Temporary
x8–x9, x18–x27	s0–s11	Saved register
x10–x17	a0–a7	Arguments / return value

Same instruction, two spellings

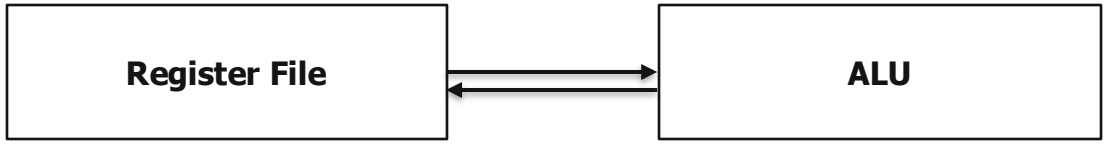
```
lui s0, 0x20000
# is exactly
lui x8, 0x20000
```

The assembler translates s0 into register number 8; the hardware never sees the name.

Load/Store Architecture

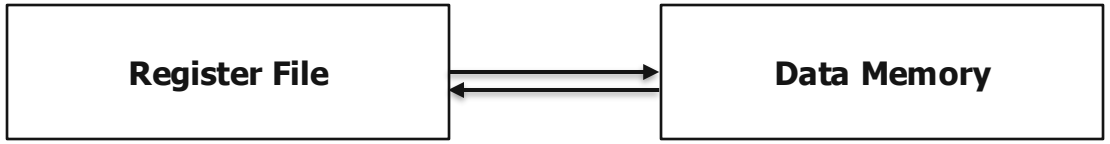
Arithmetic operates only on registers. Memory is touched only by explicit load and store instructions.

Arithmetic operates on registers




```
add x3, x1, x2
```

Memory is accessed explicitly




```
lw x3, 0(x8)    # memory → reg
sw x3, 0(x8)    # reg → memory
```

Rule	Consequence
ALU operation ↔ Registers	Operands must be loaded into registers first
Memory access ↔ LW / SW only	No instruction adds directly from memory

The 18 Instructions Our CPU Supports

Grouped by instruction format — this subset is enough for calculations, if/else, loops, function calls, memory access and I/O.

Type	Instructions	Meaning
R-type	`ADD SUB AND OR XOR SLT SLL SRL`	`rd = rs1 op rs2`
I-type · ALU	`ADDI ANDI ORI`	`rd = rs1 op imm`
I-type · Load	`LW`	`rd = Mem[rs1 + imm]`
I-type · Jump	`JALR`	`rd = PC + 4;  PC = (rs1 + imm) & ~1`
S-type	`SW`	`Mem[rs1 + imm] = rs2`
B-type	`BEQ BNE`	`if (rs1 == / != rs2)  PC = PC + imm`
U-type	`LUI`	`rd = imm << 12`
J-type	`JAL`	`rd = PC + 4;  PC = PC + imm`

03

Instruction Formats

Why this matters

Every instruction is 32 bits. Fixed field positions keep the decoder simple — but the immediate bits of B- and J-type are scrambled, a classic source of bugs.

One Instruction = 32 Bits

From these 32 bits alone, the CPU must discover everything needed to execute the instruction.

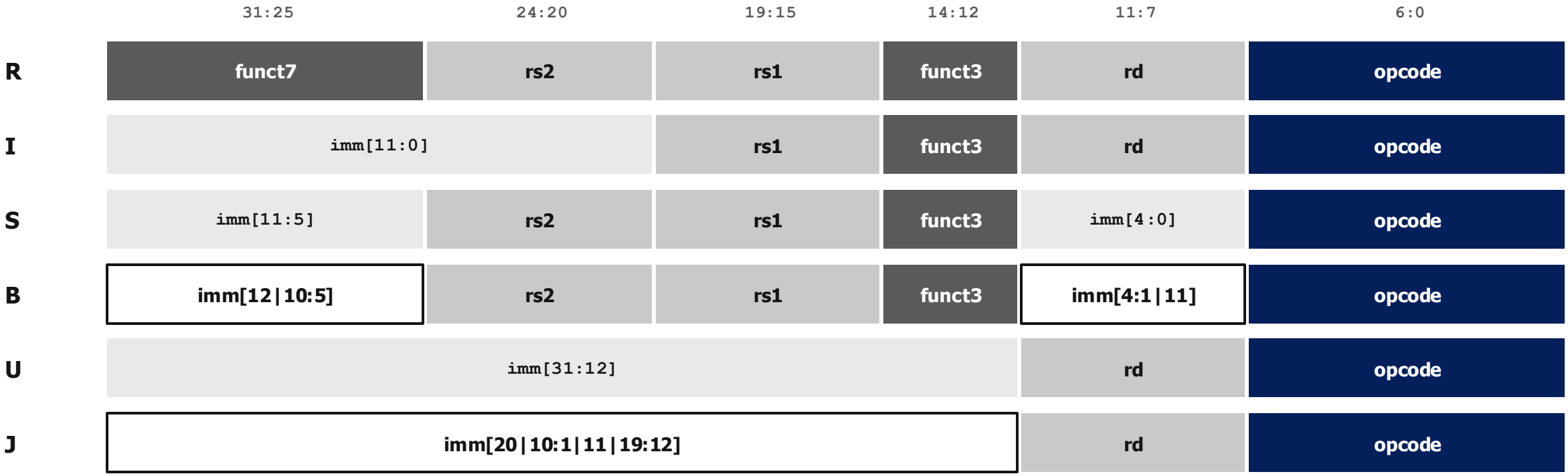
```
add x3, x1, x2
```

RISC-V answers each question with a fixed field in a structured instruction format.

The CPU must discover…	Answered by
What operation?	opcode · funct3 · funct7
Which source registers?	rs1 · rs2
Which destination register?	rd
Is there an immediate?	format, from opcode
Is memory involved?	opcode (load / store)
Should the PC change?	opcode (branch / jump)

RV32I Instruction Formats

opcode, rd, funct3, rs1, rs2 and funct7 sit in the same bits in every format that has them — the Register File can read rs1/rs2 before decode even finishes.



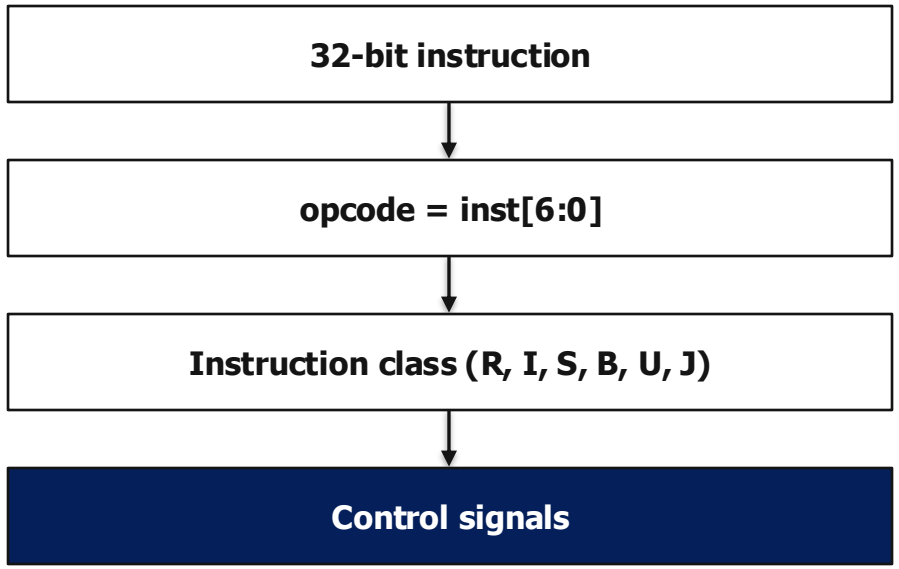
Legend navy = opcode · dark gray = funct3/funct7 · light gray = registers · pale = immediate · **outlined = scrambled immediate (B, J)**

Opcode: What Kind of Instruction Is This?

The decoder looks at inst[6:0] first — it decides the instruction class, and the class decides the control signals.

Type	opcode [6:0]
R-type	0110011
I-type ALU	0010011
Load	0000011
Store	0100011
Branch	1100011
JAL	1101111
JALR	1100111
LUI	0110111

Decoder idea



Keep in mind: R-type and I-type ALU differ only in opcode[5] — we will need this bit in Section 06.

Immediate Generator

inst[31] is always the sign bit, so sign extension can start immediately. U-type fills the low 12 bits with zeros; B and J do not store bit 0.

```
// I-type : ADDI, LW, JALR
imm_i = {{20{inst[31]}}, inst[31:20]};
// S-type : SW
imm_s = {{20{inst[31]}}, inst[31:25], inst[11:7]};
// B-type : BEQ, BNE
imm_b = {{19{inst[31]}}, inst[31], inst[7], inst[30:25], inst[11:8], 1'b0};
// U-type : LUI
imm_u = {inst[31:12], 12'b0};
// J-type : JAL
imm_j = {{11{inst[31]}}, inst[31], inst[19:12], inst[20], inst[30:21], 1'b0};
```

Format	Why the bold lines are special
B / J	Branch and jump targets are always even, so bit 0 is not stored — the reachable range doubles, but the bits must be re-ordered

COMMON AI MISTAKE

Common Mistake: B-Type and J-Type Bit Order

Each box reads: immediate bits ← taken from instruction bits. The outlined boxes are where the order jumps around.

B-type	**imm[12]** ← inst[31]	**imm[11]** ← inst[7]	**imm[10:5]** ← inst[30:25]	**imm[4:1]** ← inst[11:8]	**imm[0]** = 0
J-type	**imm[20]** ← inst[31]	**imm[19:12]** ← inst[19:12]	**imm[11]** ← inst[20]	**imm[10:1]** ← inst[30:21]	**imm[0]** = 0

Why is bit 0 not stored? Branch and jump targets are always even, so bit 0 is always 0.

Common AI mistake

AI-generated Verilog frequently gets the B/J immediate bit order wrong, or forgets sign extension. The code still compiles — only a test with a known branch or jump offset reveals it. Check every bit against the format table.

INSTRUCTOR REVEAL — AFTER VOTE

Quick Check: Which Statement Is Correct?

Think before revealing the answer.

A. Every immediate is inst[31:20]

B. B-type and J-type require rearranging instruction bits

C. Immediate values never require sign extension

D. Branch targets can have bit 0 = 1

Answer: B

A only I-type uses inst[31:20]

C signed immediates are sign-extended from inst[31]

D targets are even — bit 0 is always 0

Instructor notes: hide the answer card (Animations → Appear) until the class has voted.

04

Single-Cycle Datapath

Why this matters

We build the datapath one step at a time — fetch, decode, execute, memory, write back — and then see that all five happen inside a single clock cycle.

Now Build the Processor: Where Is the Current Instruction?

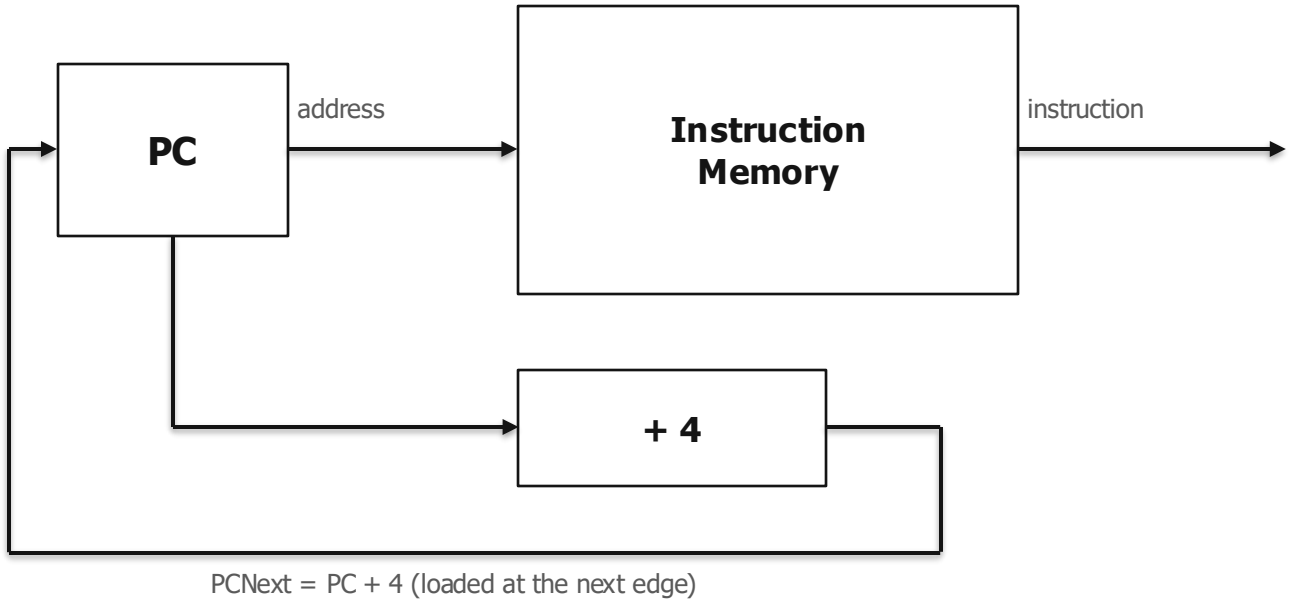
The Program Counter (PC) is a 32-bit register that stores the address of the current instruction.



Property	Meaning
Sequential	The PC changes only at the rising clock edge
Reset value	PC = 0x0000_0000 — the first instruction of the program
Byte address	The PC points to a byte; instructions are 4 bytes, so PC normally steps by 4

Step 1 — Instruction Fetch (IF)

instr = IMEM[PC]: the PC is the address, the memory returns the 32-bit instruction — a combinational read within the same cycle.

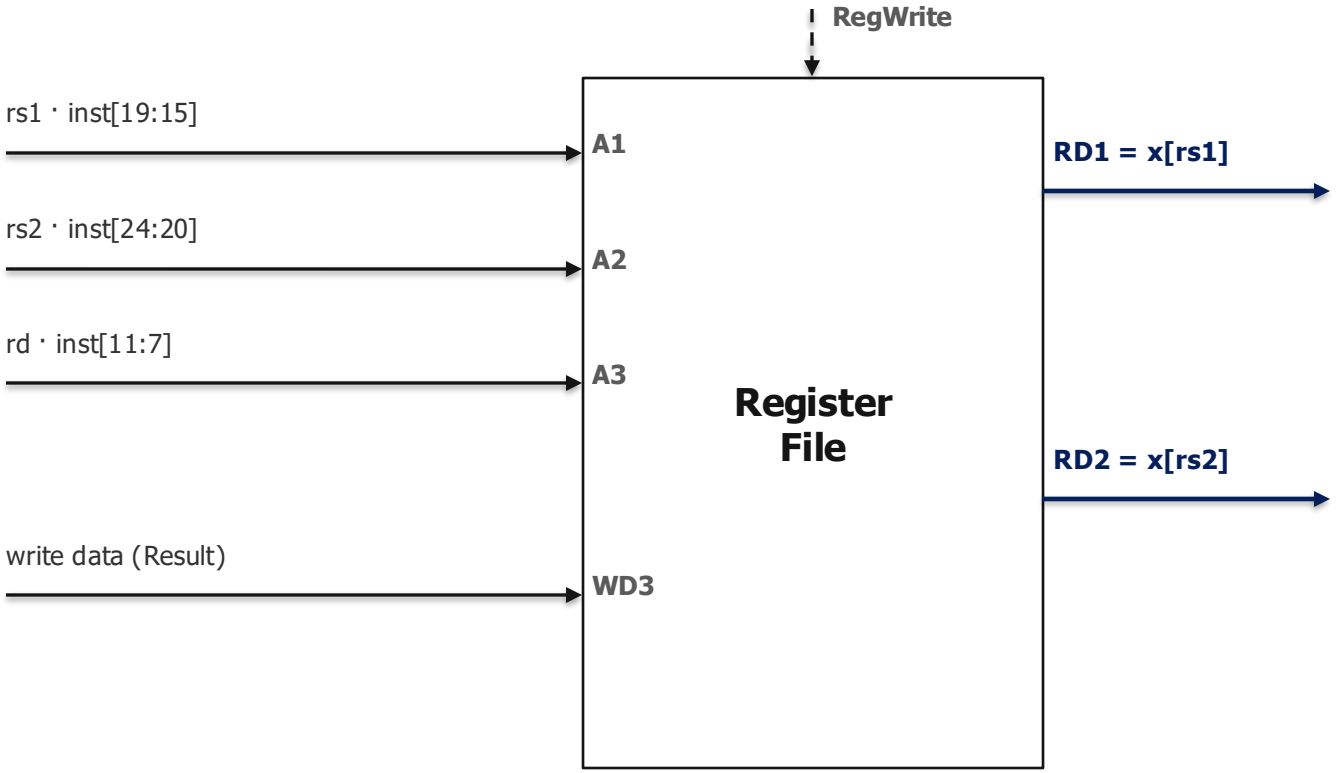


Why + 4?

- Every RV32I instruction is 4 bytes
- Memory is byte-addressed
- So the next sequential instruction is at PC + 4
- Branches and jumps will later replace PC + 4 with another target

Step 2 — Decode and Read Operands (ID)

The instruction supplies register numbers rs1, rs2 and rd; the Register File returns their values in the same cycle.



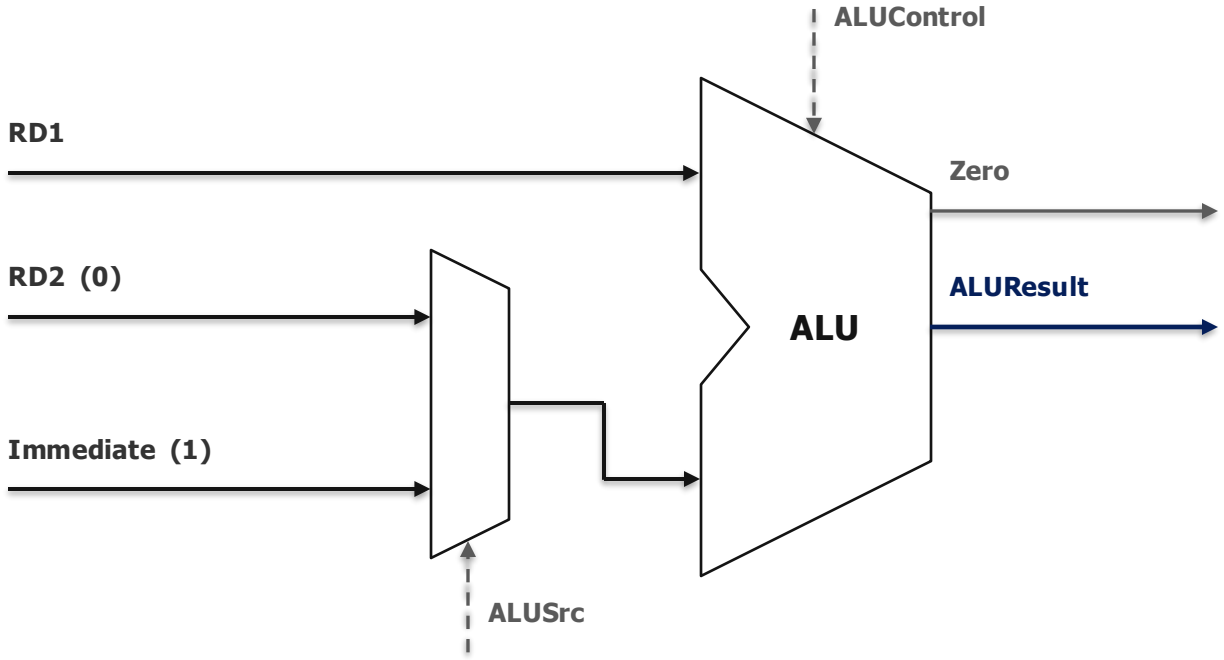
Port	Behavior
2 read ports	Combinational: RD1, RD2 in the same cycle
1 write port	Writes rd at the rising edge, only if RegWrite = 1

Hardware rule

x0 must always read as zero. Writes to x0 must be ignored.

Step 3 — Execute (EX): ALU and ALUSrc

Operand B of the ALU comes from RD2 (R-type, BEQ/BNE) or from the immediate (ADDI, LW, SW, JALR) — so we need a multiplexer.



ALU operations

ADD	SUB	AND	OR
XOR	SLT	SLL	SRL

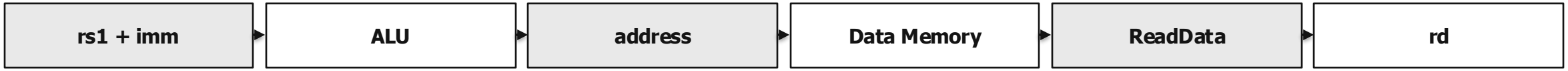
Zero flag

Zero = 1 when the ALU result is 0. BEQ and BNE use it to decide whether to branch.

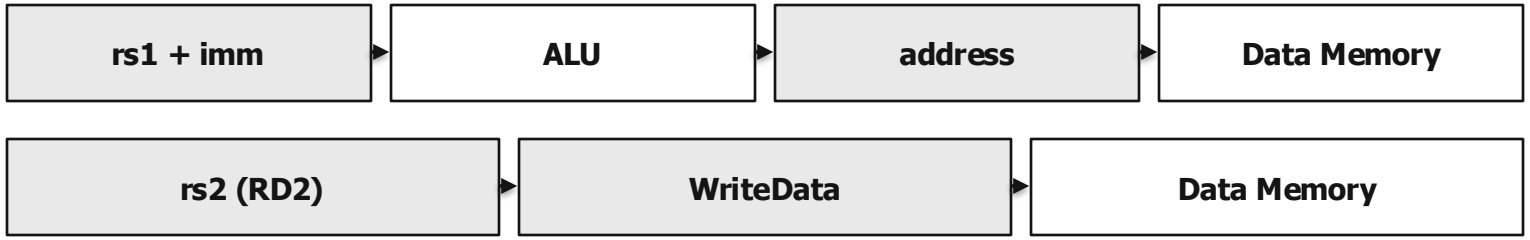
Step 4 — Memory Access (MEM)

For LW and SW the ALU result is not a final answer — it is a memory address.

Load `lw rd, imm(rs1)`



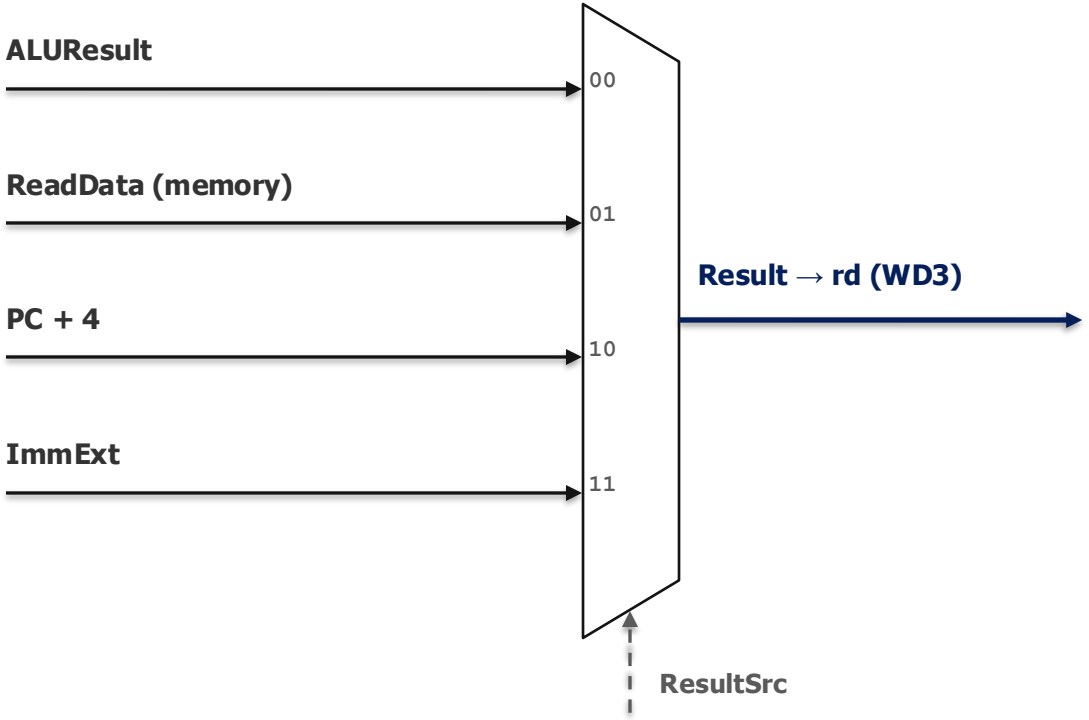
Store `sw rs2, imm(rs1)`



Access	Timing in the single-cycle CPU
LW reads	Combinational — data is ready within the same cycle
SW writes	At the rising clock edge, only when MemWrite = 1

Step 5 — Write Back (WB): the Result MUX

What value is written to rd depends on the instruction — four possible sources, selected by ResultSrc.



ResultSrc	Writes	Used by
00	ALU result	R-type, I-type ALU
01	Memory data	LW
10	PC + 4	JAL, JALR
11	Immediate	LUI

Five Logical Operations — One Clock Cycle

IF, ID, EX, MEM and WB are names for what happens — not pipeline stages. In this processor all five occur combinationally within one cycle.

Step	Name	What happens
IF	Instruction Fetch	Read instruction at PC from IMEM
ID	Instruction Decode	Decode, read rs1/rs2, build immediate
EX	Execute	ALU: result, address or comparison
MEM	Memory Access	lw reads / sw writes Data Memory
WB	Write Back	Write the result into rd

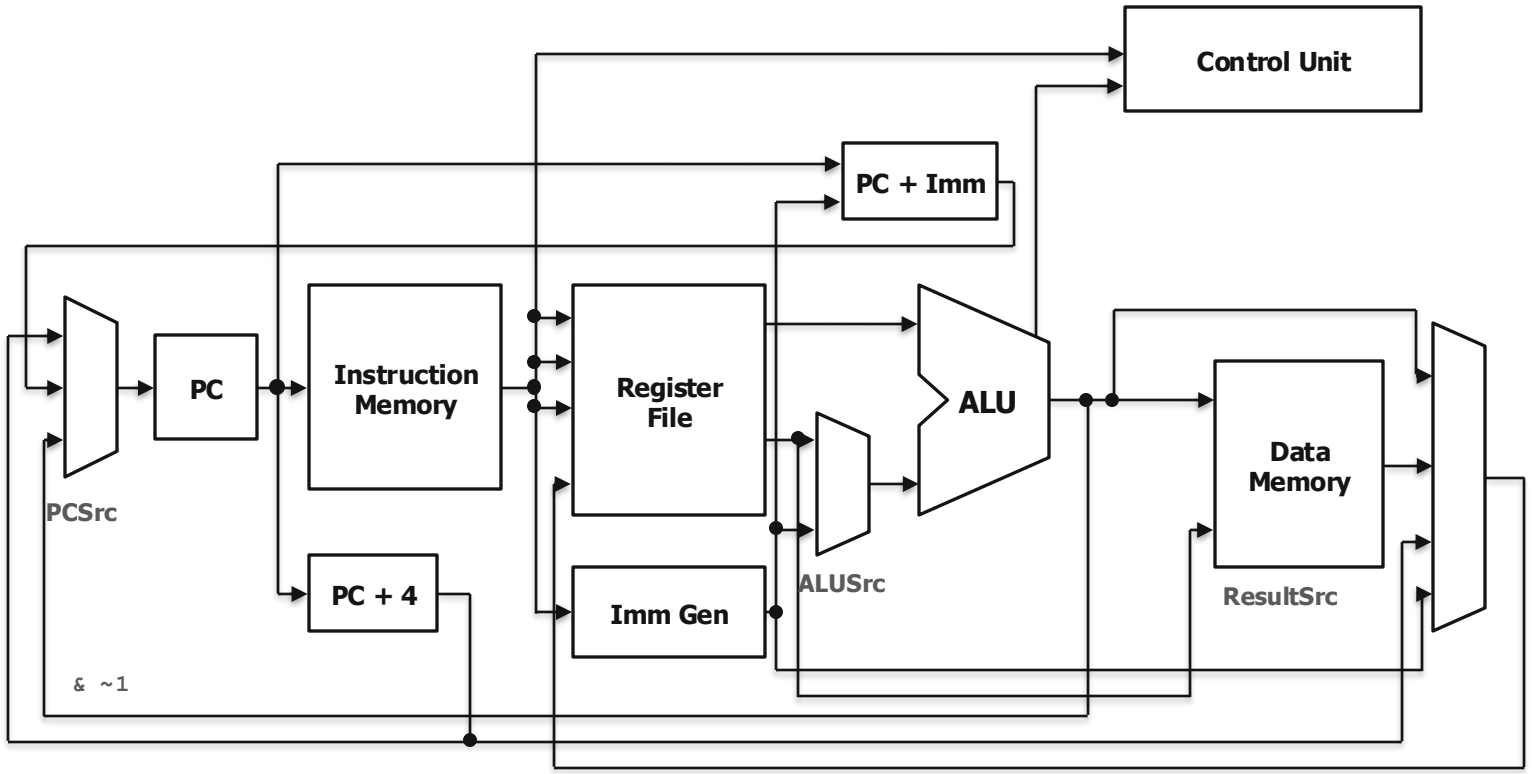
Important

These are NOT pipeline stages. The whole path settles within one clock period; PC, registers and memory update together at the next rising edge.



The Complete Single-Cycle RV32I Datapath

Three MUXes choose between sources — PCSrc, ALUSrc and ResultSrc — and two adders compute PC + 4 and the branch/JAL target PC + Imm.



What to notice

MUX	Chooses
PCSrc	PC+4, PC+Imm or JALR target
ALUSrc	RD2 or immediate
ResultSrc	ALU, memory, PC+4 or imm

Control outputs drive every MUX select and write enable, those wires are not drawn to keep the data paths readable.

05

Follow Real Instructions Through the Datapath

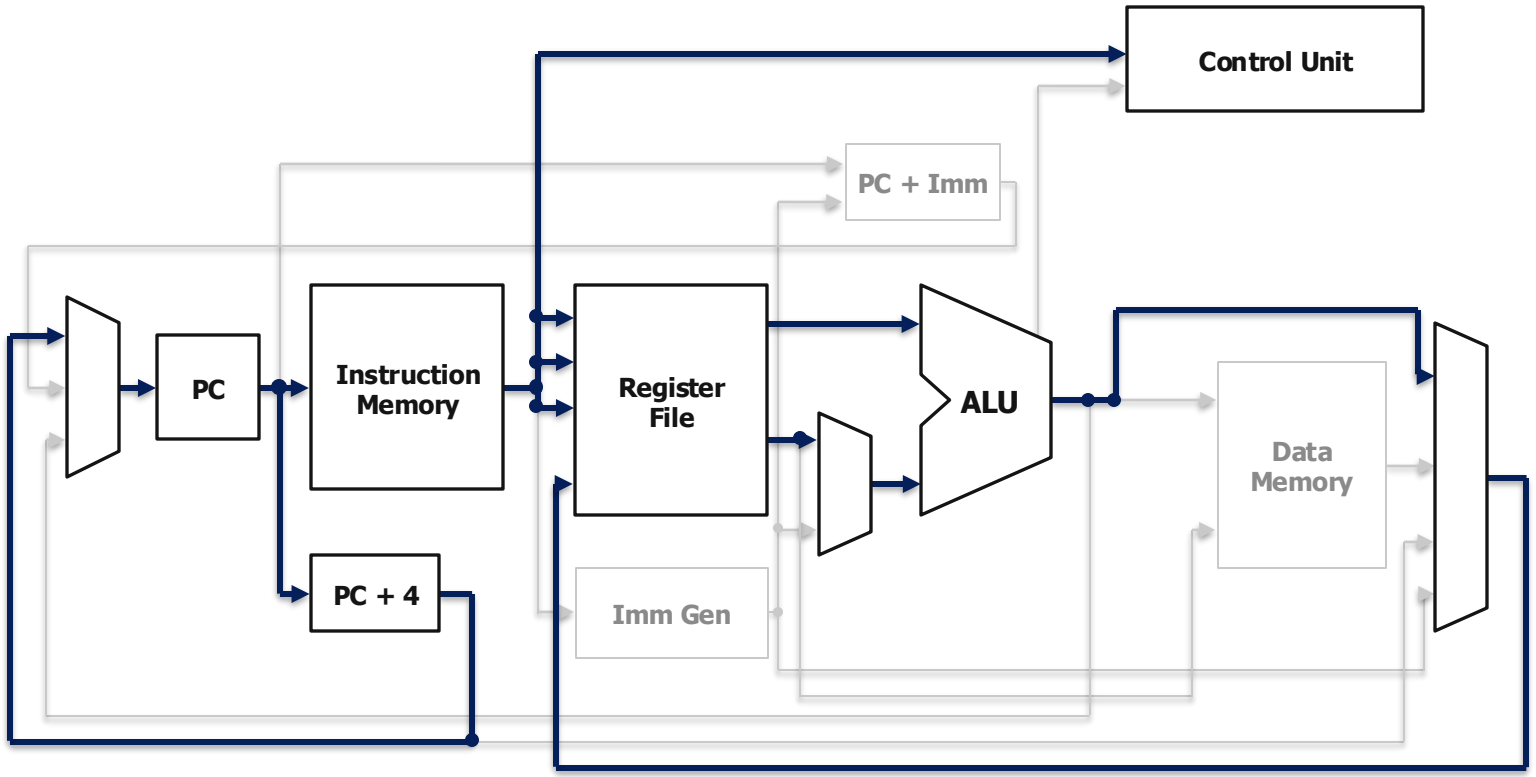
Why this matters

The same hardware serves every instruction. Tracing six instructions shows which parts each one uses — and which control signals must be set.

ADD — R-type

```
add x3, x1, x2
```

Register to register: two reads, one ALU operation, one write back.



Active path

1. Read x1 → RD1, x2 → RD2
2. ALUSrc = 0: ALU takes RD2
3. ALU adds: 5 + 10 = 15
4. Result MUX selects ALU
5. Write 15 into x3 at the clock edge

Control signals

Signal	Value
RegWrite	1
ALUSrc	0
MemWrite	0
ResultSrc	00
ALUOp	10
PCSrc	0

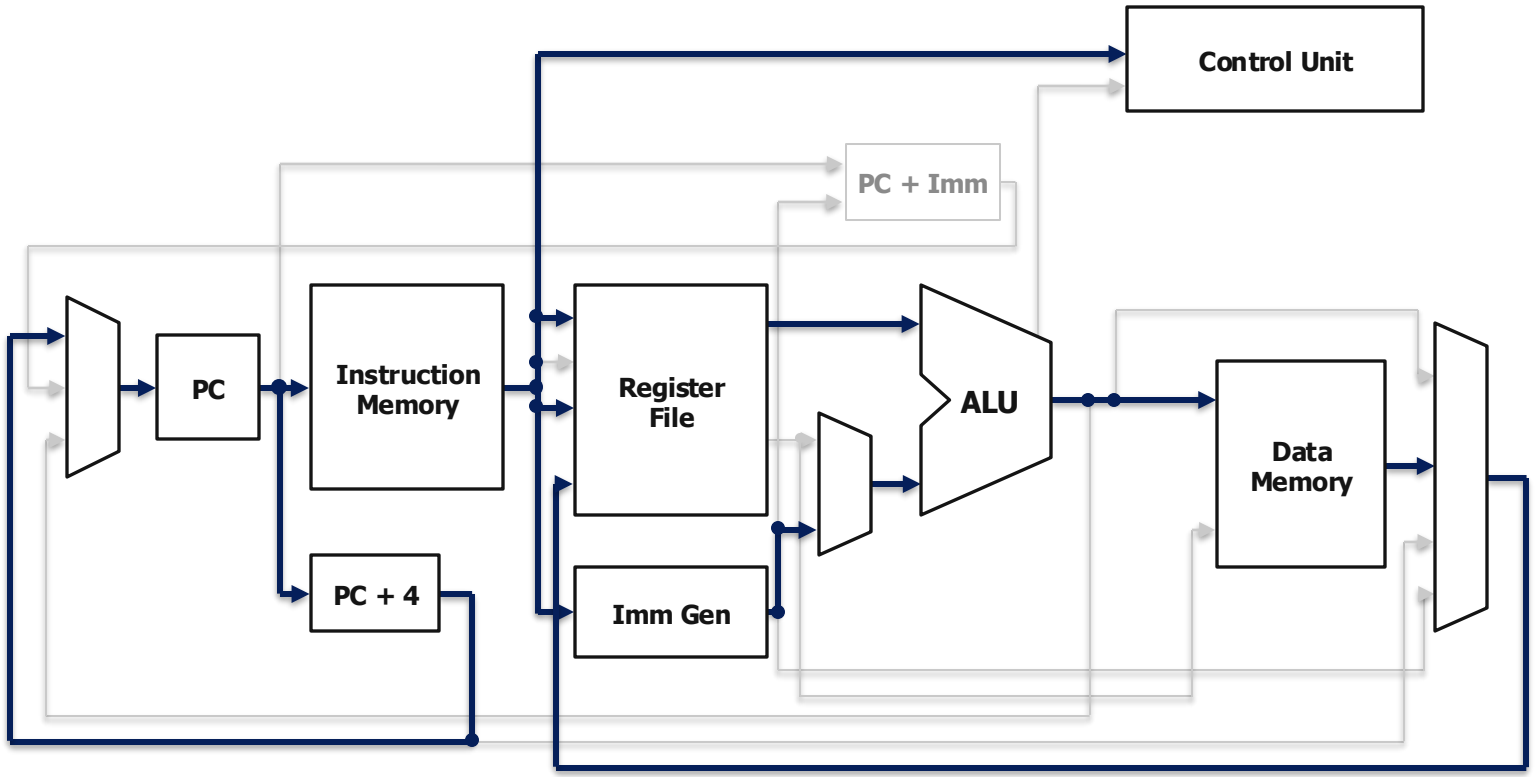
State change

x3 ← 15 · PC ← PC + 4. Data Memory, Imm Gen and PC + Imm are unused.

LW — the Longest Path

`lw x9, 0(x8)`

A load uses every block in the datapath — which is why it sets the clock period.



Active path

1. Read x8 = 0x1000_0000
2. Imm Gen: I-type imm = 0
3. ALU: x8 + 0 → address
4. DMEM reads that address
5. ResultSrc = 01: x9 ← data

Control signals

Signal	Value
RegWrite	1
ImmSrc	I
ALUSrc	1
MemWrite	0
ResultSrc	01
ALUOp	00

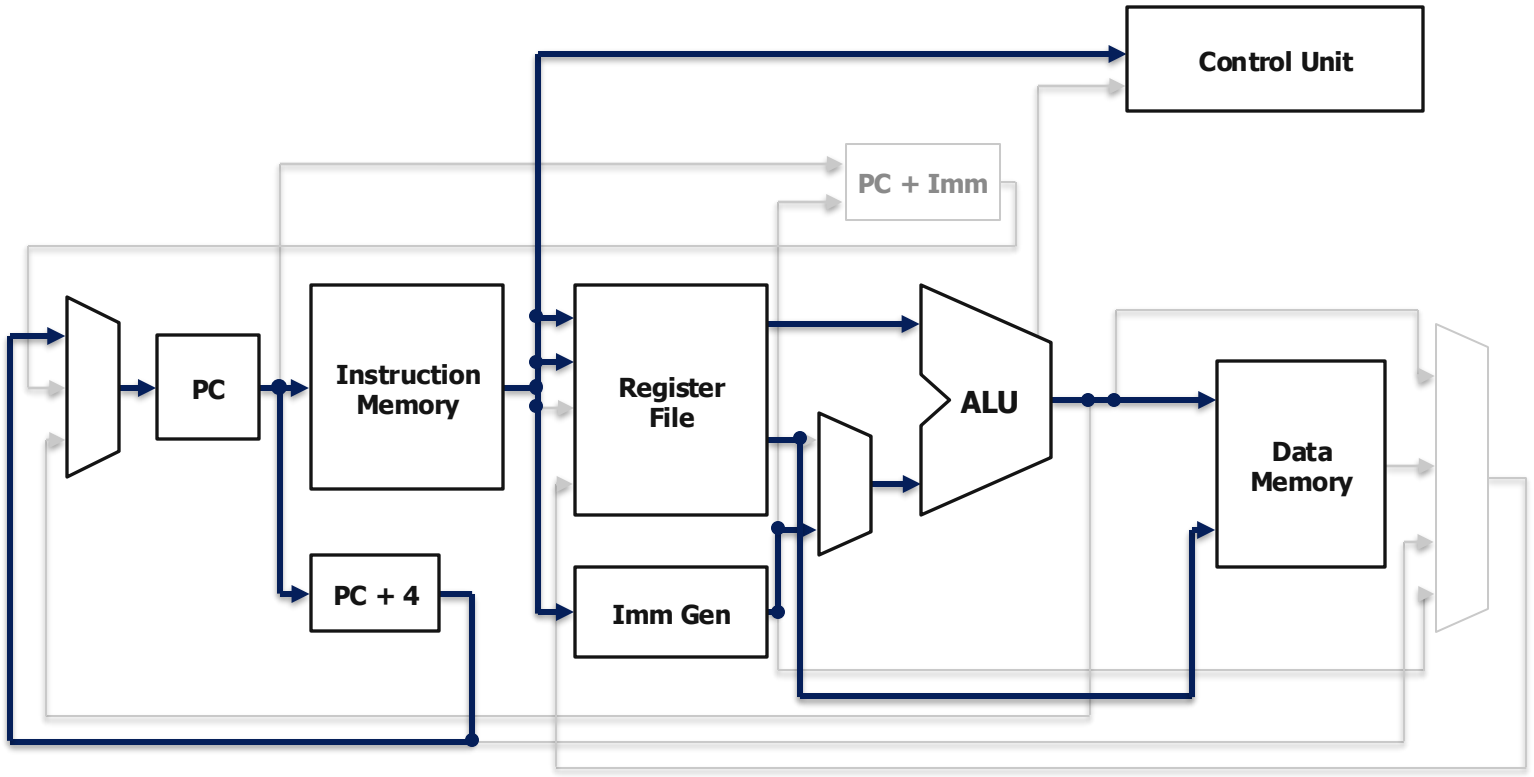
State change

x9 ← Mem[x8] = 15 · PC ← PC + 4.

SW — Address Path + Data Path

`sw x3, 0(x8)`

Two paths at once: the ALU forms the address while RD2 carries the data to memory.



Active path

1. Read x8 (base) and x3 (data)
2. Imm Gen: S-type imm = 0
3. ALU: x8 + 0 → address
4. RD2 = x3 → WriteData
5. MemWrite = 1: write at the edge

Control signals

Signal	Value
RegWrite	0
ImmSrc	S
ALUSrc	1
MemWrite	1
ResultSrc	–
ALUOp	00

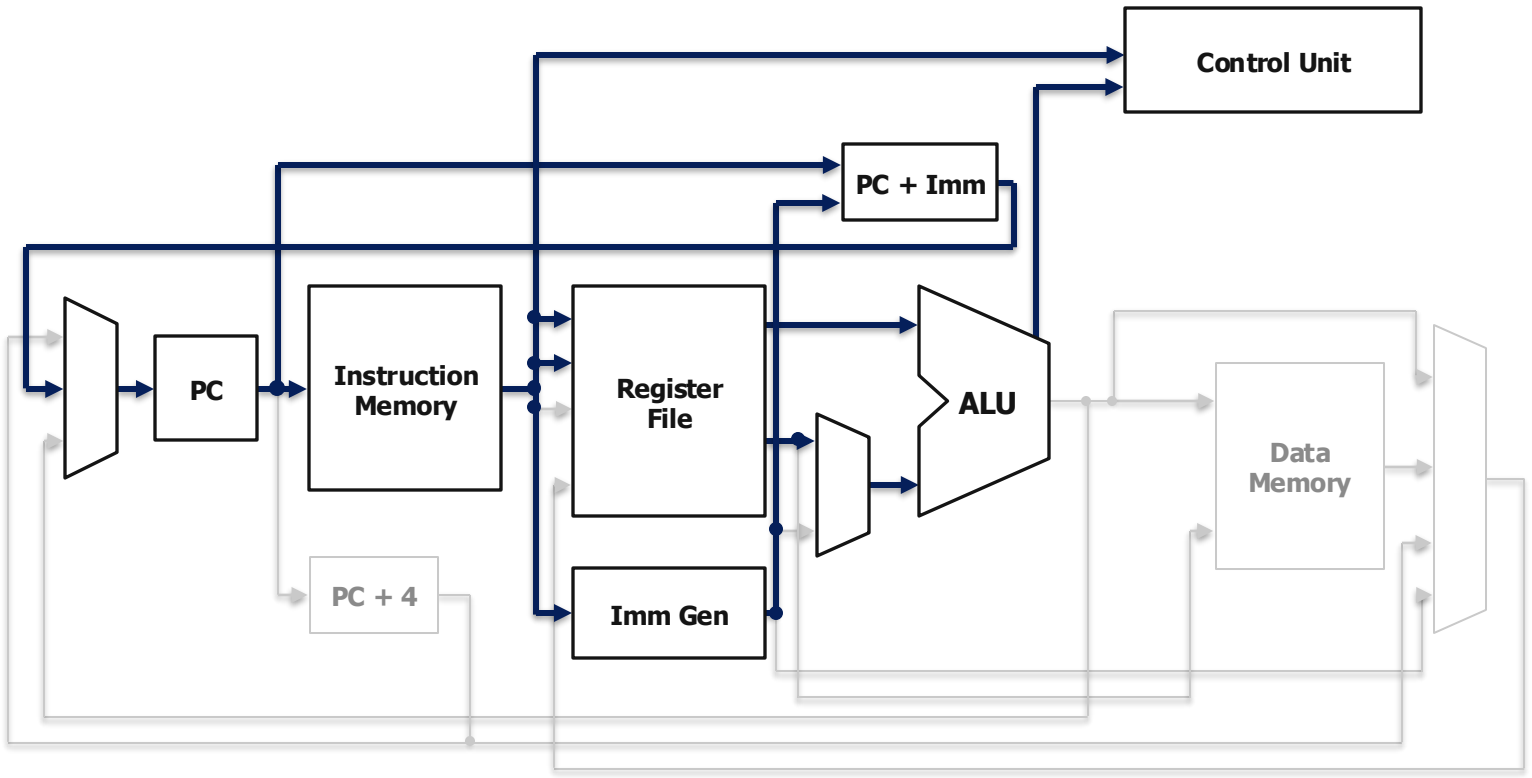
State change

Mem[0x1000_0000] ← 15 · no register changes — SW changes memory, not rd.

BEQ — Compare + Target

```
beq x9, x3, skip
```

At PC = 0x2C the ALU compares while a separate adder computes the target — both in the same cycle.



Active path

1. Read x9 and x3 (15, 15)
2. ALU SUB: 15 − 15 = 0 → Zero = 1
3. PC + Imm: 0x2C + 8 = 0x34
4. branch_taken = 1 → PCSrc
5. No register or memory write

Control signals

Signal	Value
RegWrite	0
ImmSrc	B
ALUSrc	0
MemWrite	0
Branch	1
ALUOp	01

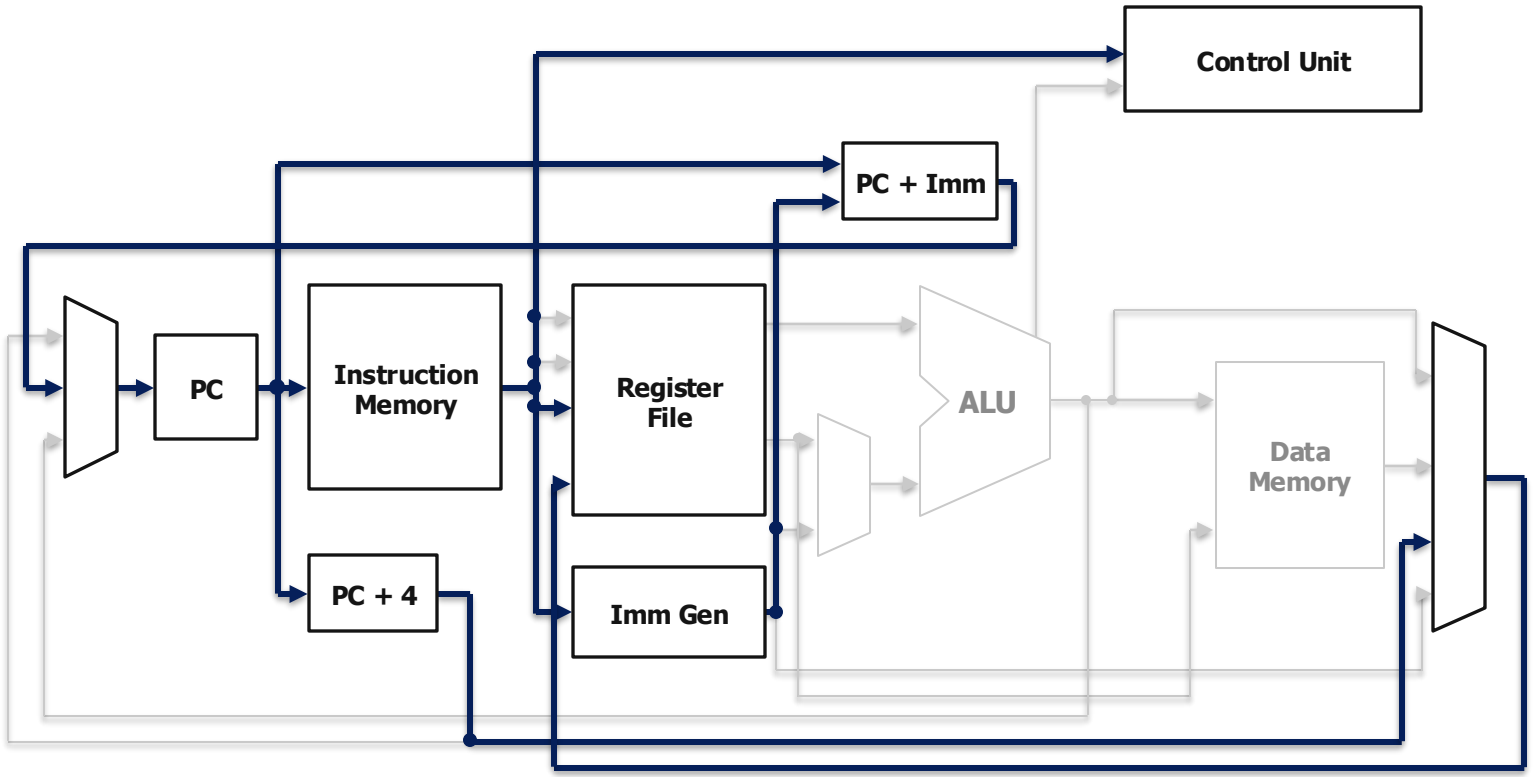
State change

PC ← 0x34 — the instruction at 0x30 is skipped.

JAL — Link + Jump

`jal x11, func`

At PC = 0x34 JAL does two things at once: save the return address and jump.



Active path

1. Link: PC + 4 = 0x38
2. ResultSrc = 10: x11 ← 0x38
3. Imm Gen: J-type imm = 0x14
4. Jump: 0x34 + 0x14 = 0x48
5. Jump = 1: PCSrc takes the target

Control signals

Signal	Value
RegWrite	1
ImmSrc	J
ResultSrc	10
MemWrite	0
Jump	1
ALUSrc	–

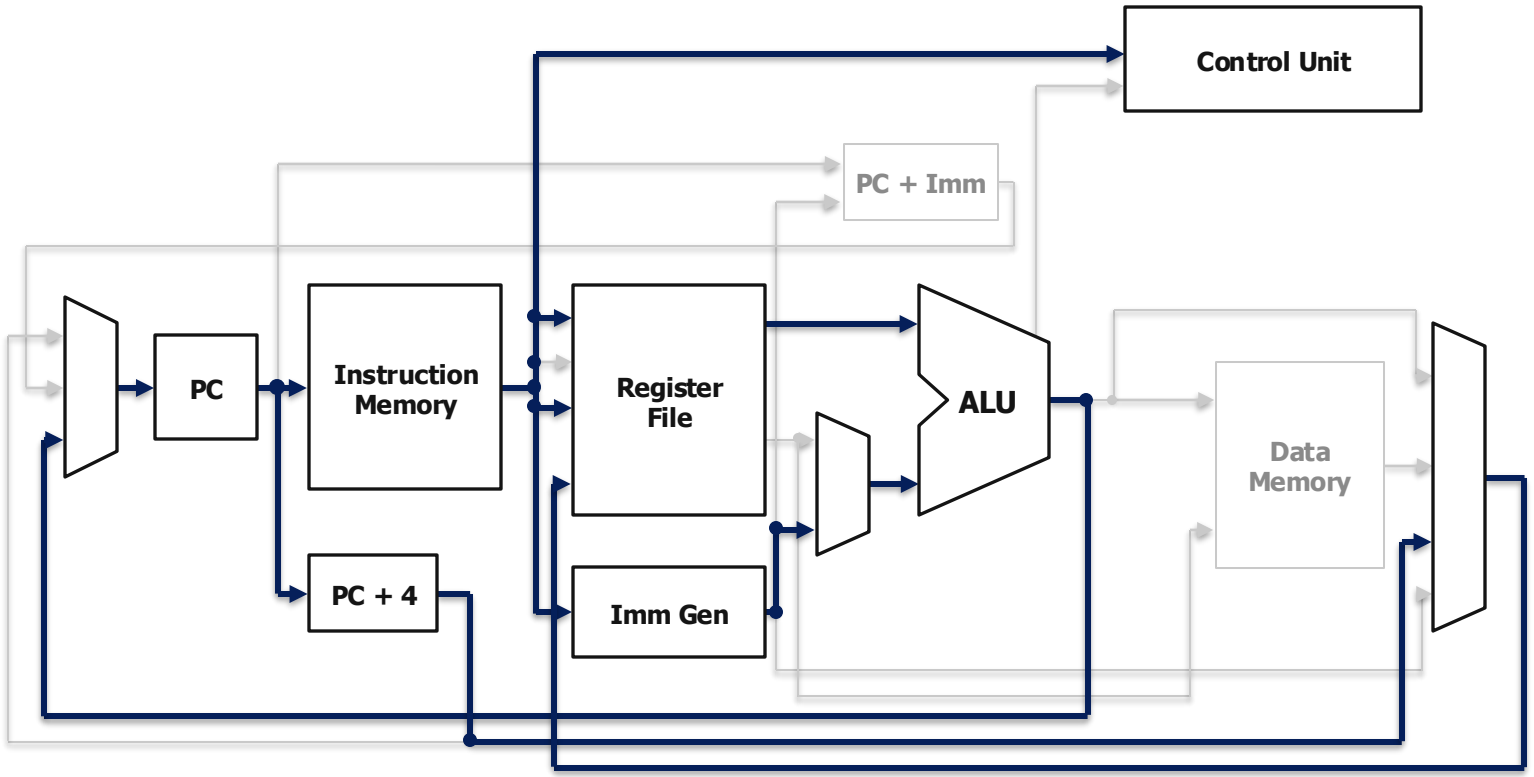
State change

x11 ← 0x38 (return address) · PC ← 0x48 (func). ALU and Data Memory are unused.

JALR — Jump to a Register

```
jalr x0, 0(x11)
```

At PC = 0x4C the target comes from a register, computed by the ALU, with bit 0 cleared.



Active path

1. Read x11 = 0x38
2. ALU: 0x38 + 0 = 0x38
3. & ~1 clears bit 0 → 0x38
4. PCSrc selects the JALR target
5. PC + 4 = 0x50 → x0: ignored

Control signals

Signal	Value
RegWrite	1
ImmSrc	I
ALUSrc	1
ResultSrc	10
Jump	1
ALUOp	00

State change

PC ← 0x38 (return) · x0 stays 0 — writing the link to x0 turns JALR into a plain return.

06

Control Unit

Why this matters

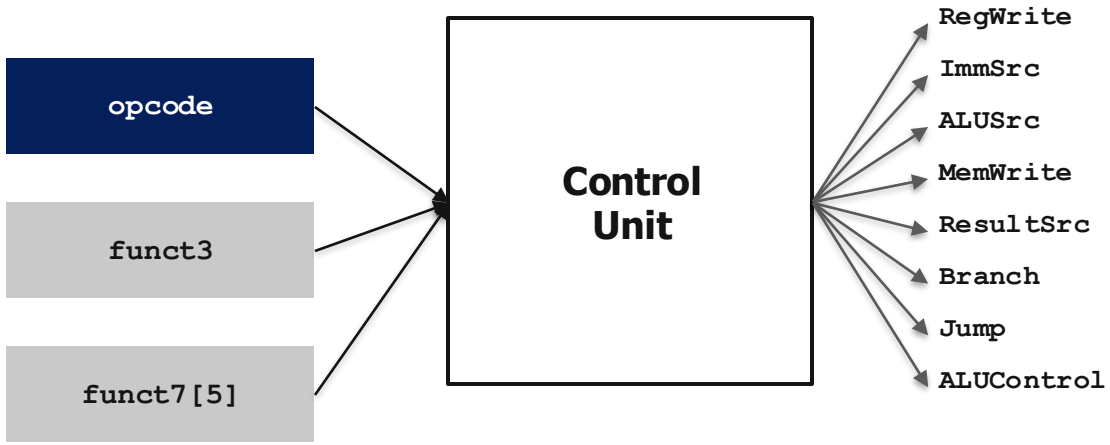
The datapath has many possible paths but cannot choose between them. The control unit turns instruction bits into the signals that select the right path.

Who Controls the Datapath?

The same wires serve every instruction — the datapath alone cannot know what to do. The Control Unit is purely combinational: instruction bits in, control signals out.

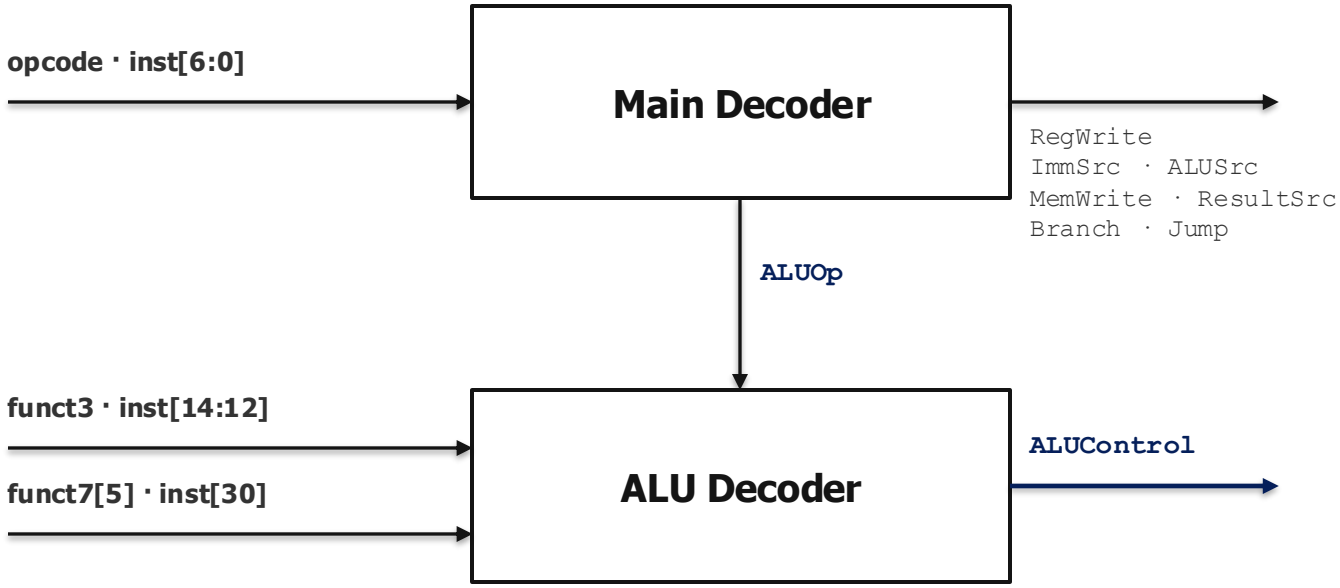
For each instruction, decide:

- Write a register?
- Use RD2 or the immediate?
- Write memory?
- Select ALU or memory result?
- Branch? Jump?
- Which ALU operation?



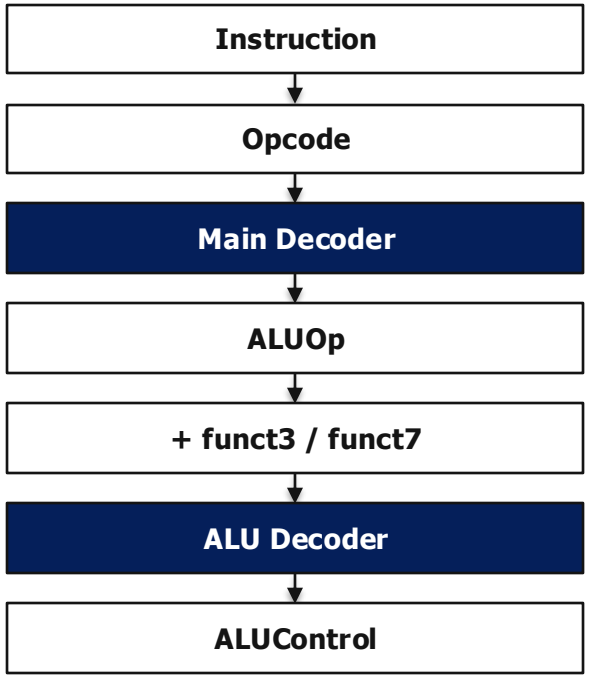
Two Decoders: Main Decoder + ALU Decoder

The Main Decoder finds the instruction class from the opcode; the ALU Decoder works out the exact ALU operation from ALUOp, funct3 and funct7[5].



SUB only for R-type: the ALU Decoder also checks opcode[5]

Decoding hierarchy



What Do the Control Signals Mean?

Control signals are not abstract — each one is a wire into a MUX select or a write enable.

Signal	Drives this hardware	Meaning
`RegWrite`	Register File write enable	1 = write rd at the clock edge
`ImmSrc`	Immediate Generator	Choose format: I / S / B / U / J
`ALUSrc`	MUX before ALU input B	0 = RD2, 1 = immediate
`MemWrite`	Data Memory write enable	1 = write Data Memory
`ResultSrc`	Result MUX	00 ALU · 01 Memory · 10 PC+4 · 11 Immediate
`Branch`	Next-PC logic	Instruction is BEQ / BNE
`Jump`	Next-PC logic	Instruction is JAL / JALR
`ALUOp`	ALU Decoder	00 add · 01 subtract (branch) · 10 look at funct
`ALUControl`	ALU	The exact ALU operation

Main Decoder Table

– means don't care in the table — but Verilog must still assign a safe default to every output, otherwise synthesis infers a latch.

Instr.	opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	Jump	ALUOp
R-type	0110011	1	–	0	0	00	0	0	10
I-type ALU	0010011	1	I	1	0	00	0	0	10
LW	0000011	1	I	1	0	01	0	0	00
SW	0100011	0	S	1	1	–	0	0	00
BEQ/BNE	1100011	0	B	0	0	–	1	0	01
JAL	1101111	1	J	–	0	10	0	1	–
JALR	1100111	1	I	1	0	10	0	1	00
LUI	0110111	1	U	–	0	11	0	0	–

Notice

Only SW writes memory; SW and BEQ/BNE are the only instructions with RegWrite = 0.

ALU Decoder Table

ALUOp tells the ALU Decoder how much to look at: 00 / 01 are fixed operations; 10 means decode funct3 (and funct7[5]).

ALUOp	funct3	Condition	Operation
00	–	– (LW, SW, JALR)	ADD
01	–	– (BEQ, BNE)	SUB
10	000	R-type and funct7[5] = 1	SUB
10	000	otherwise (ADD, ADDI)	ADD
10	111	–	AND
10	110	–	OR
10	100	–	XOR
10	010	–	SLT (signed)
10	001	–	SLL
10	101	–	SRL

The highlighted row

This is where most decoder bugs live: SUB must be selected only for an R-type instruction with funct7[5] = 1.

SLT

Must compare signed: −1 < 5 is true.

A Dangerous Decoder Bug: ADDI Becomes SUB

In an I-type instruction inst[30] is part of the immediate — but it sits exactly where R-type keeps funct7[5].

```
addi x1, x0, -1024
```

imm = −1024 = **1100 0000 0000**



Wrong

```
sub = (ALUOp == 2'b10) && (funct3 == 3'b000)
      && funct7_5;
```

Correct — also check R-type

```
sub = (ALUOp == 2'b10) && (funct3 == 3'b000)
      && opcode[5] && funct7_5;
```

Verification lesson

With funct3 = 000, a decoder that only checks funct7[5] turns this ADDI into a SUB. Never test only positive immediates.

INSTRUCTOR REVEAL — AFTER ANSWERS

Quick Control Challenge: sw x5, 8(x2)

Set every control signal before revealing the answer.

Signal	Your answer
RegWrite	?
ImmSrc	?
ALUSrc	?
MemWrite	?
ALUOp	?

Answer

RegWrite 0 — SW changes memory, not rd
ImmSrc S
ALUSrc 1 — address = x2 + 8
MemWrite 1
ALUOp 00 → ADD
Data x5 travels on RD2 to WriteData

Instructor notes: hide the answer card until students have filled the table.

07

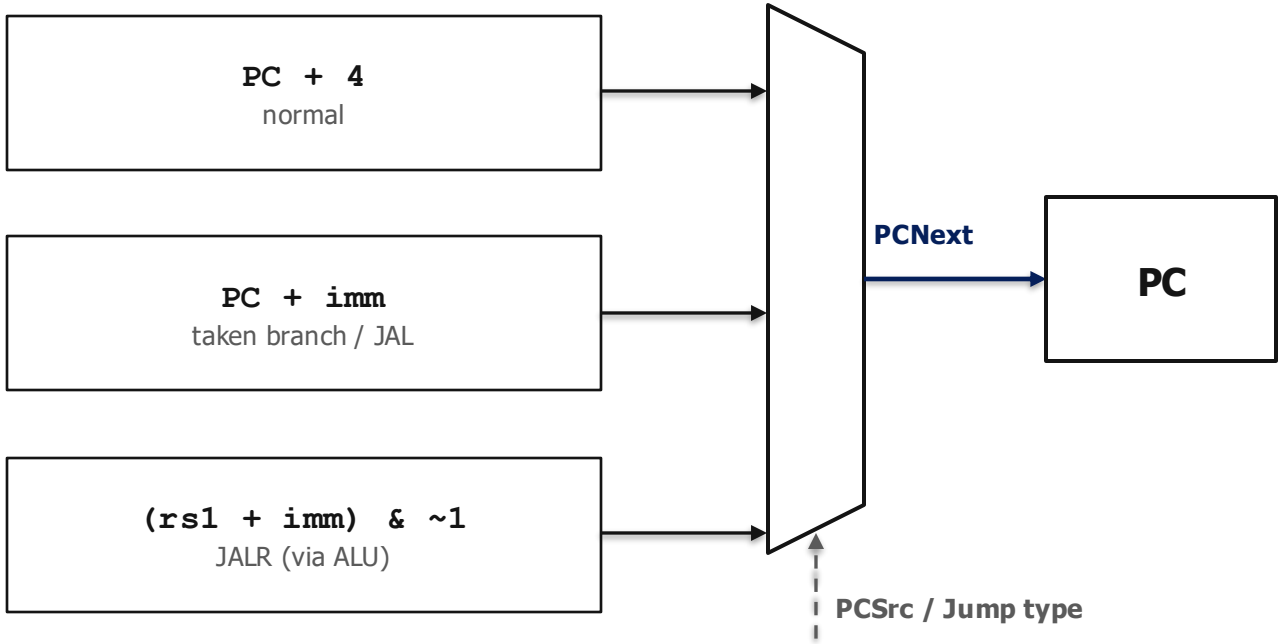
Next-PC Logic and Timing

Why this matters

Branches and jumps change where the next instruction comes from. And because every instruction takes one cycle, the slowest one sets the clock for all.

Where Does the Next PC Come From?

Three possible sources feed the Next-PC MUX; the PC loads the selected value at the next rising edge.



Instruction	PCNext
normal	`PC + 4`
BEQ / BNE taken	`PC + imm_b`
BEQ / BNE not taken	`PC + 4`
JAL	`PC + imm_j`
JALR	`(rs1 + imm_i) & ~1`

Branch Decision: BEQ and BNE

The ALU subtracts rs1 − rs2 (ALUOp = 01); equal operands give zero = 1. The decision (ALU) and the target (PC + imm adder) are computed in the same cycle by separate hardware.

```
assign branch_taken = Branch & ((funct3 == 3'b000) ?  zero   // BEQ
                                                    : ~zero); // BNE
assign PCSrc        = branch_taken | Jump;
```

Instr.	funct3	ALU zero	branch_taken
BEQ	000	1 (equal)	1 — take branch
BEQ	000	0	0 — PC + 4
BNE	001	0 (not equal)	1 — take branch
BNE	001	1	0 — PC + 4

Key point

BEQ and BNE share one datapath. Only funct3 decides whether zero or ~zero means "taken".

INSTRUCTOR REVEAL — AFTER ANSWERS

PC Challenge: Target of a Taken Branch

Current instruction address PC = 0x0000_0020. Instruction: beq x1, x2, -8 — assume the branch is taken.

What is the next PC?

Answer

Calculation 0x0000_0020 + (−8) = 0x0000_0018

Why The target is the PC of the branch itself plus imm_b — not PC + 4 plus the offset

INSTRUCTOR — ASK FIRST

Single-Cycle: CPI = 1 — But Is It Fast?

Every instruction completes in one clock cycle, so CPI = 1. But the clock period must be long enough for the slowest instruction — and every other instruction waits just as long.

CPI = 1

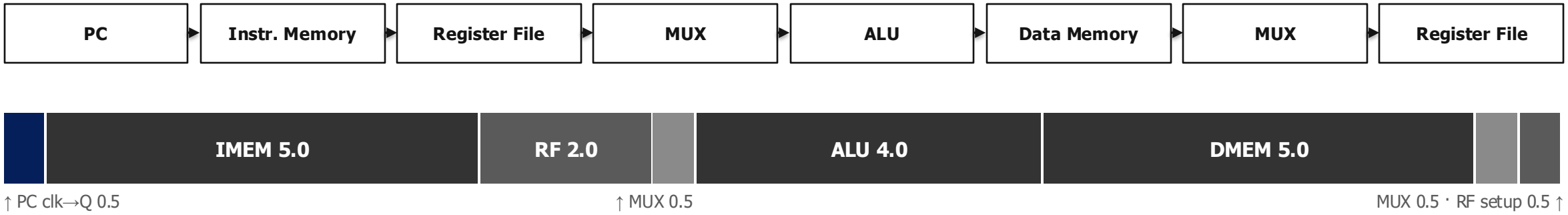
Which instruction uses the longest path?

LW — it passes through every block:

Instruction Memory, Register File, ALU, Data Memory and the write back to the Register File.

The LW Critical Path

Tc(single) = t_pcq + t_IMEM + t_RFread + t_mux + t_ALU + t_DMEM + t_mux + t_RFsetup — example values for teaching.



Component	ns	Component	ns	Result
PC clock-to-Q	0.5	ALU	4.0	Total ≈ 18.0 ns
Instruction Memory	5.0	Data Memory	5.0	Fmax ≈ 55 MHz
Register File read	2.0	MUX	0.5	
MUX	0.5	Register File setup	0.5	

An add skips Data Memory entirely — yet it must still wait the full 18 ns clock period.

CPI = 1 Does Not Mean “Fast”

Tc ≈ 18 ns, so Fmax ≈ 1 / 18 ns ≈ 55 MHz. Execution Time = Instruction Count × CPI × Clock Period

Advantages	Limitations	Motivates (later chapters)
Simple architecture	Every instruction waits for the slowest path (LW)	Multi-cycle processors
Easy to understand	Two memories and extra adders needed	Pipelined processors
No pipeline hazards		

```
Execution Time = Instruction Count × CPI × Clock Period
```

We use these architectures only as motivation — this chapter stays single-cycle.

08

Memory Architecture

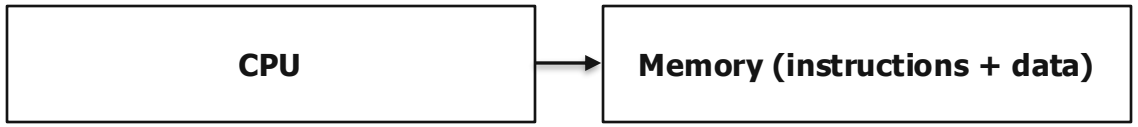
Why this matters

A single-cycle CPU forces a memory choice, and byte addressing forces an indexing rule. Both are easy to get wrong in Verilog.

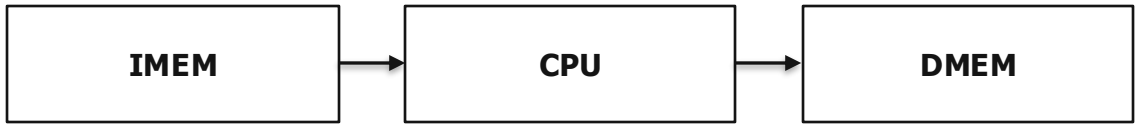
Von Neumann vs Harvard

During LW or SW, the CPU must fetch the instruction and access data memory in the same clock cycle — one memory port cannot do both.

Von Neumann



Harvard — our CPU



	Von Neumann	Harvard
Memory	One memory for instructions and data	Separate IMEM and DMEM
Simultaneous fetch + data	Limited	Yes
Advantage	Economical, flexible	Fetch and load/store at the same time
Used in	Main memory of general CPUs	Single-cycle CPUs, DSPs, L1 caches

Why Harvard here

Single-cycle needs Harvard: the instruction fetch and the data access both happen within one cycle.

Byte Addressing and Word Alignment

PC holds a byte address, but a Verilog memory array is indexed by words — drop the two low address bits.

address	+0	+1	+2	+3	word index
0x000	byte	byte	byte	byte	imem[0]
0x004	byte	byte	byte	byte	imem[1]
0x008	byte	byte	byte	byte	imem[2]
0x00C	byte	byte	byte	byte	imem[3]

- 1 address = 1 byte · 1 word = 4 bytes
- Aligned word address: addr[1:0] = 00
- Word-wide memory is indexed with addr[N:2]

Example: PC = 0x008

```
PC       = ...0000 1000
PC[11:2] = ...0000 10 = 2
```

Correct

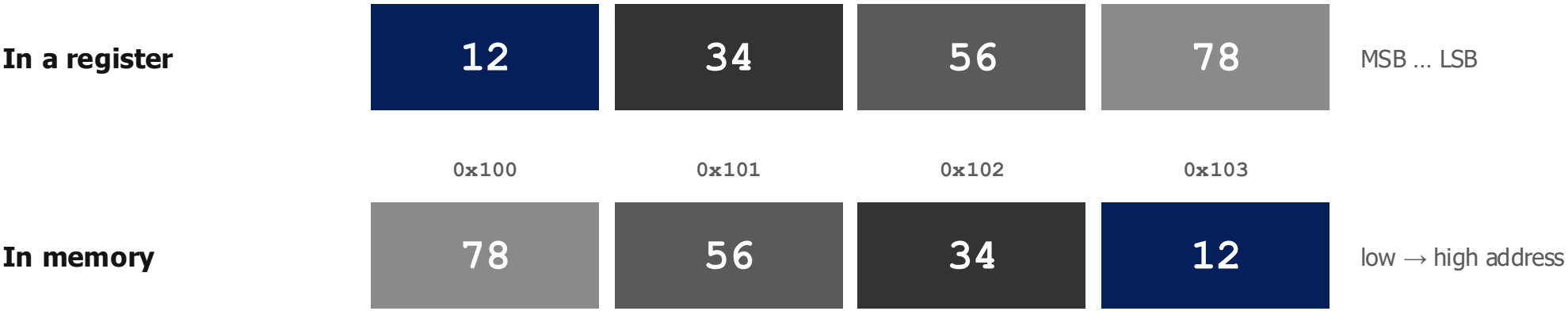
```
reg [31:0] imem [0:1023];  // 4 KB
assign instr = imem[pc[11:2]];
```

Incorrect

```
assign instr = imem[pc];   // reads imem[8]!
```

Little-Endian

The least-significant byte is stored at the lowest address. Store 0x12345678 starting at address 0x100:



Address	Byte
0x100	78
0x101	56
0x102	34
0x103	12

09

Bus, Memory Map and Address Decoding

Why this matters

Our CPU can calculate and use memory. To talk to LEDs, switches and timers without new instructions, we give every device an address on a shared bus.

INSTRUCTOR REVEAL — ASK FIRST

Our CPU Can Calculate. Now What?

We can execute instructions and read and write memory. A useful embedded system must also talk to the outside world.

LEDs	**Switches**	**Buttons**	**7-segment**
Timers	**UART**	**Sensors**	**Accelerators**

Do we need a new CPU instruction for every device?

No.

The CPU already has lw and sw — we only need to give each device an address.

Reuse the Same Interface

The CPU already knows how to move data with lw and sw. What if peripherals also appear at addresses?

Today: memory

```
lw → read memory
sw → write memory
```

Same instructions: peripherals

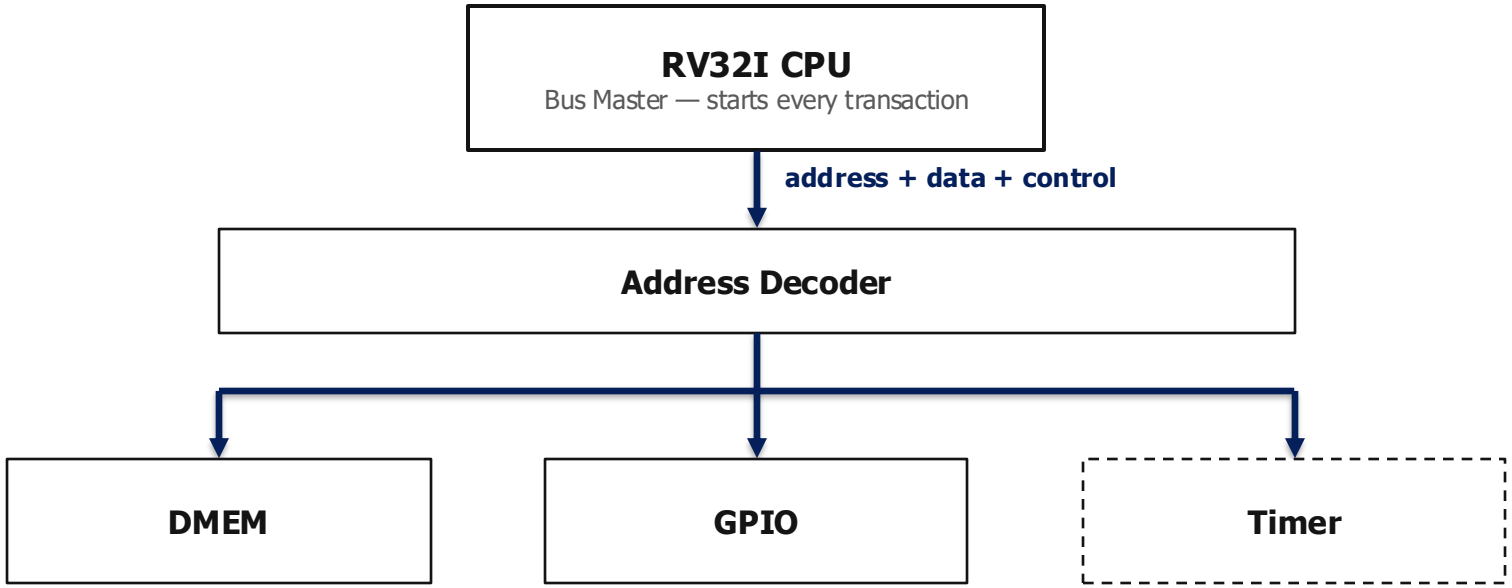
```
lw → read peripheral
sw → write peripheral
```

This is the foundation of Memory-Mapped I/O

But first we need a shared communication path between the CPU and every device — a bus.

What Is a Bus? Masters and Slaves

A bus is one shared set of signals connecting one master to many devices — instead of separate wires to each device.



Bus Slaves — each responds only when its address is selected

Bus Master
The CPU. It initiates every read and write.

Bus Slave
Memory or a peripheral. It responds when its address is selected.

Key point
The CPU does not need to know whether an address is RAM or an LED register.

Basic Bus Signals

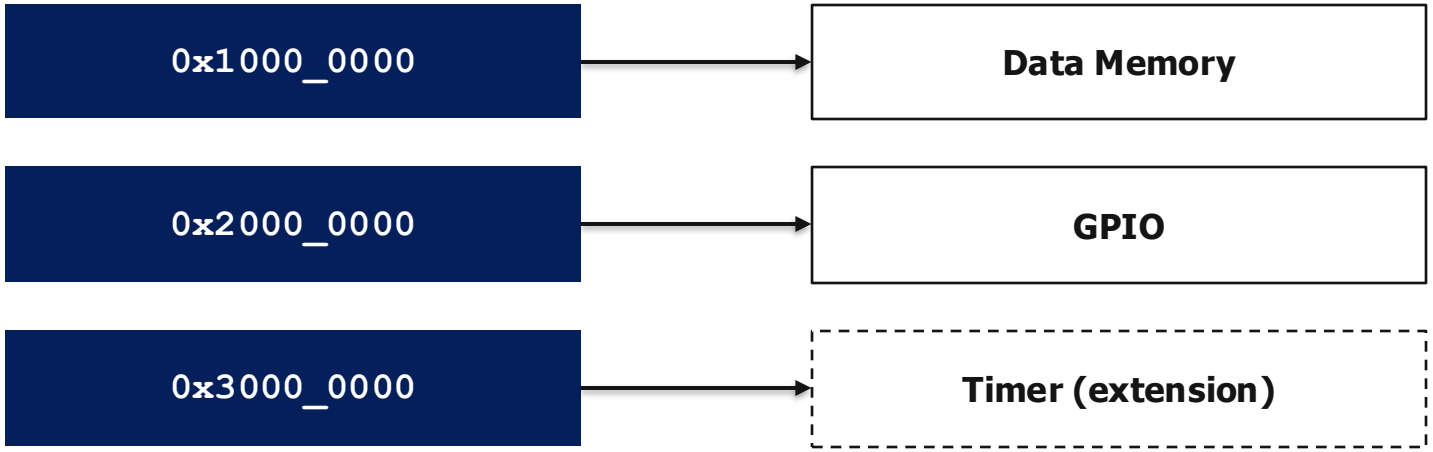
In our single-cycle CPU, addr, wdata and we leave the CPU combinationally; writes happen at the rising edge when we = 1 and the device is selected; read data must arrive within the same cycle.

Signal	Direction	Meaning
`addr[31:0]`	CPU → Slave	Address bus — which location / device
`wdata[31:0]`	CPU → Slave	Data to write
`rdata[31:0]`	Slave → CPU	Data read back
`we`	CPU → Slave	Write enable
`re`	CPU → Slave	Read enable (optional in single-cycle)
`wstrb[3:0]`	CPU → Slave	Byte enables for SB/SH (extension)

Industry buses — AMBA APB/AHB/AXI (ARM), Wishbone (open source), TileLink (RISC-V/SiFive) — use the same address + data + control idea; they differ in handshake and speed.

One Bus, Many Devices → the Memory Map

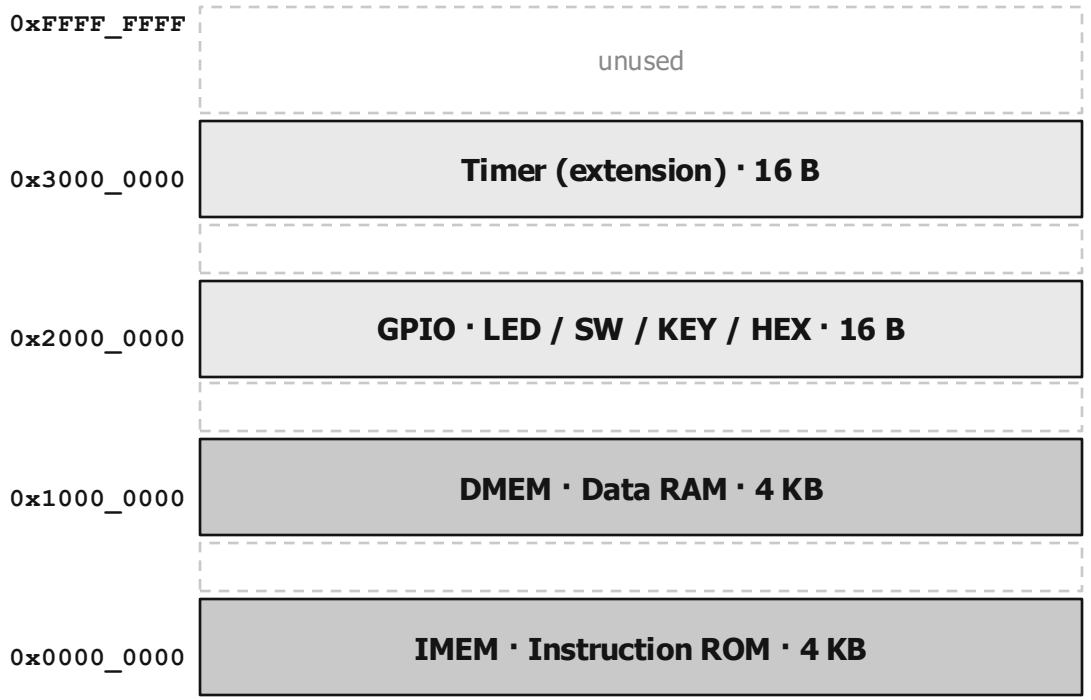
The CPU places an address on the bus; the system decides which device owns that address. This assignment is called the Memory Map.



Memory map
A table that assigns every region of the 4 GB (2^{32}) address space to one device.

Our Memory Map: 4 GB of Addresses

IMEM is reached through the separate instruction port; DMEM, GPIO and Timer are reached through the data bus. Not to scale.



Term	Meaning
Base address	First address of a region, e.g. GPIO = 0x2000_0000
Address region	The range a device owns: base + size
Peripheral register	One addressable word inside a device, e.g. LED at offset 0x0
Address decoder	Looks at the upper address bits and selects the device

Address Decoder: Creating Chip Selects

The decoder looks at the top 4 address bits and raises exactly one chip select.

```
always @(*) begin
  sel_dmem = (addr[31:28] == 4'h1);
  sel_gpio = (addr[31:28] == 4'h2);
  sel_tmr  = (addr[31:28] == 4'h3);
end
```

Example

addr = **0x2000_0004**

addr[31:28] = 0010 = 2

sel_gpio = 1

sel_dmem = 0, sel_tmr = 0. The low bits (0x4) then pick the register inside GPIO.

Never Write to the Wrong Device

A global we from the CPU is not enough — it would write every device at once. Gate it with each chip select.

```
assign we_dmem = we & sel_dmem;
assign we_gpio = we & sel_gpio;
assign we_tmr  = we & sel_tmr;
```

If no device is selected

For example sw to 0x4000_0000: no device may be written. tb_soc.v checks this case.

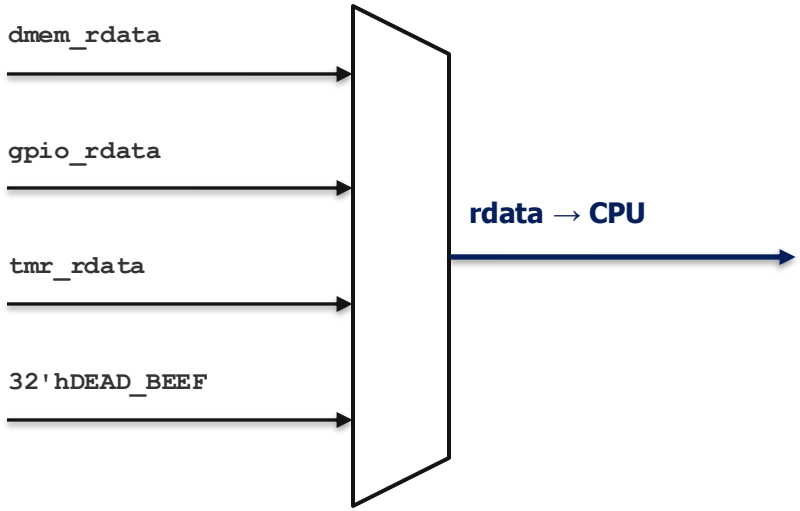
Rule

- A device may write only when the CPU requests a write
- … and that device is selected

Read-Data MUX

The CPU has one rdata input, but many devices can answer. Select by chip select; return an obvious pattern for unmapped addresses.

```
always @(*) begin
  if      (sel_dmem) rdata = dmem_rdata;
  else if (sel_gpio) rdata = gpio_rdata;
  else if (sel_tmr)  rdata = tmr_rdata;
  else               rdata = 32'hDEAD_BEEF;
end
```



Why DEAD_BEEF?

An unmapped read returns a pattern you recognise instantly in a waveform while debugging.

Full vs Partial Address Decoding

Our simple decoder compares only addr[31:28], so it treats the entire 0x2xxx_xxxx region (256 MB) as GPIO.

	Full decoding	Partial decoding
Compares	Every address bit that matters	Only some bits
Logic	More	Less
Addresses	Each address → one device	Several addresses → same device
Risk	More hardware	Address aliasing

```
addr[31:28] == 4'h2
```

= partial decoding: the lower 28 bits are ignored when choosing the device.

Address Aliasing

Suppose GPIO uses addr[31:28] to select GPIO and addr[3:2] to select a register — all other bits are ignored.

address	addr[31:28]	addr[3:2]	register
0x2000_0000	2 → GPIO	00	LED
0x2000_0010	2 → GPIO	00	LED
0x2ABC_0000	2 → GPIO	00	LED

Multiple addresses → the same hardware register

Is aliasing a bug?
It is acceptable in embedded systems — as long as the memory map documents it clearly.

10

Memory-Mapped I/O

Why this matters

For the CPU, everything is memory. A store to the right address turns on an LED; a load from another reads the switches — no special I/O instruction exists.

For the CPU, Everything Is Memory

Device registers get addresses, just like memory. No special I/O instruction — only ordinary LW and SW.

Offset	Address	Register	R/W	Connected to
0x00	0x2000_0000	LED	W	LEDG[9:0]
0x04	0x2000_0004	SW	R	SW[9:0]
0x08	0x2000_0008	KEY	R	3 push buttons on DE0
0x0C	0x2000_000C	HEX	W	HEX0–HEX3 (16-bit value)

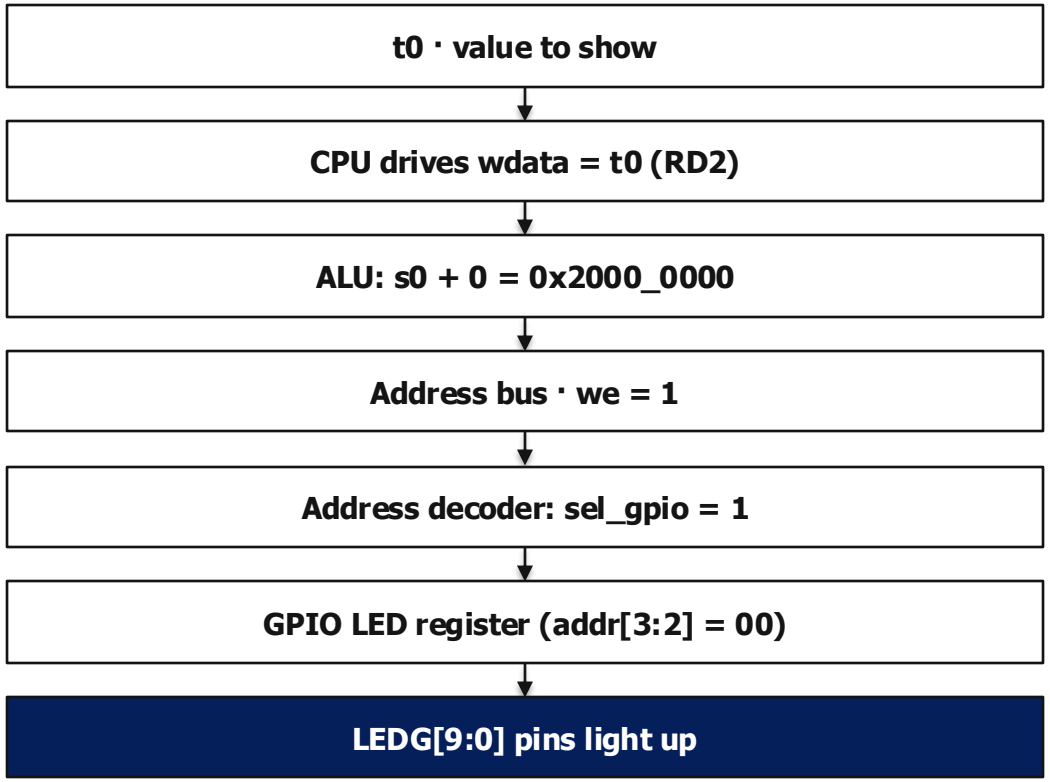
Polling
The CPU reads I/O status repeatedly in a loop — simple, and used in this chapter. Interrupts (the device notifies the CPU) are the more advanced alternative.

What Happens When We Write an LED?

sw t0, 0(s0) with s0 = 0x2000_0000 is an ordinary store — the same hardware path as sw x3, 0(x8) into DMEM.

```
sw t0, 0(s0)
```

Software instruction → physical hardware change
The only thing that makes this store an LED write is the address.



GPIO Output Register

An SW instruction ultimately writes a flip-flop — and that flip-flop drives a physical FPGA pin.

```
always @(posedge clk) begin
  if (rst)
    led_reg <= 10'b0;
  else if (we_gpio && addr[3:2] == 2'b00)
    led_reg <= wdata[9:0];
end

assign LEDG = led_reg;
```

Term	Role
`we_gpio`	CPU writes and decoder selected GPIO
`addr[3:2]=00`	Offset 0x0 — the LED register
`wdata[9:0]`	The value from the sw instruction
`rst`	Known start value: LEDs off

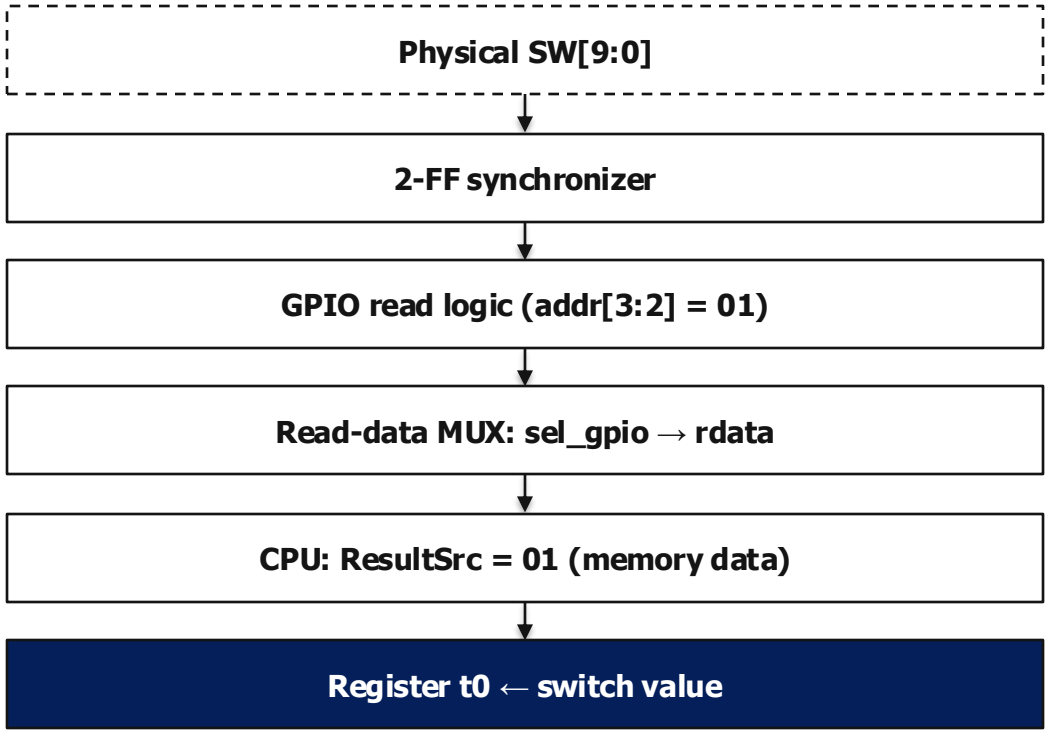
What Happens When We Read a Switch?

lw t0, 4(s0) with s0 = 0x2000_0000: the CPU sees the switches as data stored at 0x2000_0004.

```
lw t0, 4(s0)
```

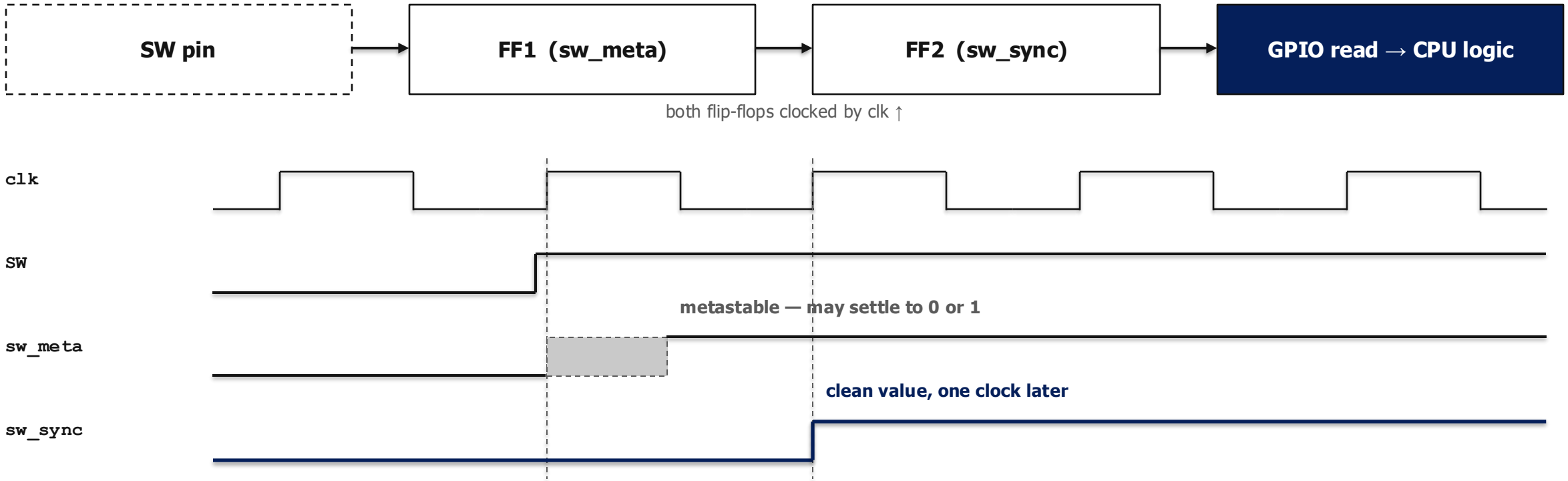
Same path as a load from DMEM

Only the chip select differs: sel_gpio instead of sel_dmem.



Why Not Connect a Switch Directly?

A switch changes independently of the clock — it is asynchronous. Sampling it right at a clock edge can cause metastability.



Synchronizer in Verilog

sw_meta is the first stage and may briefly be metastable — never use it directly. sw_sync is the only version synchronous logic should see.

```
always @(posedge clk) begin
  sw_meta <= SW;
  sw_sync <= sw_meta;
end
```

DE0 board notes

- Push buttons are active-low: pressed = 0
- 7-segment displays are active-low too
- Check pin names against the board .qsf — some templates use KEY, others BUTTON

11

FPGA Reality and Assembly to Hardware

Why this matters

Textbook single-cycle assumes combinational memory; FPGA block RAM is synchronous. And software must still be turned into bits inside instruction memory.

Textbook Memory vs FPGA Memory

Single-cycle theory expects combinational read (data in the same cycle); Cyclone III M9K block RAM gives synchronous read (data one cycle later).

Single-cycle theory expects

Combinational read — data in the same cycle

M9K block RAM gives

Synchronous read — data one cycle later

Approach	Advantage	Limitation
Small memory built from logic (this lab)	Matches single-cycle theory	Uses many logic elements
Clock memory on the falling edge	Can use M9K block RAM	Half a cycle left — lower Fmax
Change the architecture	Better FPGA fit (multi-cycle / pipeline)	No longer pure single-cycle

Lab Memory Choice

Engineering lesson: architecture is shaped by the physical technology it is built on.

IMEM

256 words

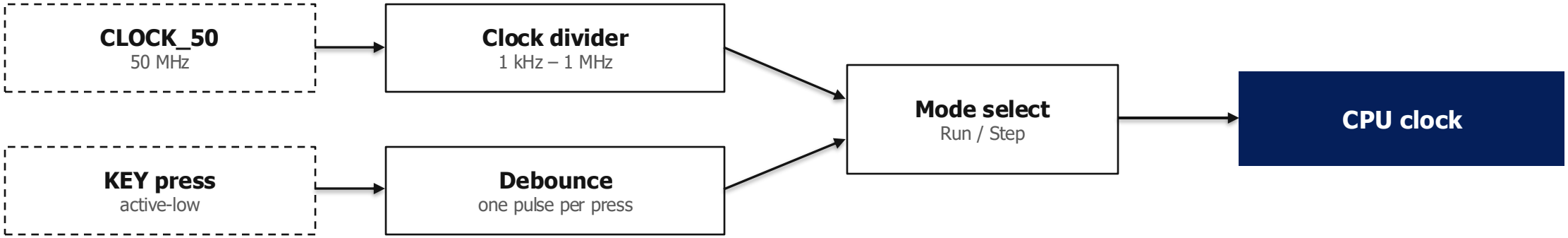
DMEM

256 words

- Small enough for Quartus to build from logic instead of block RAM
- Keeps single-cycle behavior clear: read data arrives within the same cycle

FPGA Clock and Reset

DE0 provides CLOCK_50 = 50 MHz — far too fast to watch LEDs change. Two ways to clock the CPU, plus a defined reset.



Run mode

Divided clock — the program runs continuously at a visible speed.

Step mode

One button press ≈ one instruction. Needs debounce. Ideal for debugging.

Reset

PC = 0x0000_0000 and every GPIO register starts at a known value.

How Does Assembly Reach the CPU?

Software finally becomes bit patterns stored in hardware memory.



Assembly	test.hex line
`addi x1, x0, 5`	`00500093`
`addi x2, x0, 10`	`00A00113`
`add  x3, x1, x2`	`002081B3`

Loading it in Verilog

```
initial $readmemh("test.hex", imem);
```

RARS Workflow: test.s → test.hex

Our PC resets to 0x0000_0000 and IMEM starts there, so the program's first instruction must be at address 0 — otherwise every jump target is off.

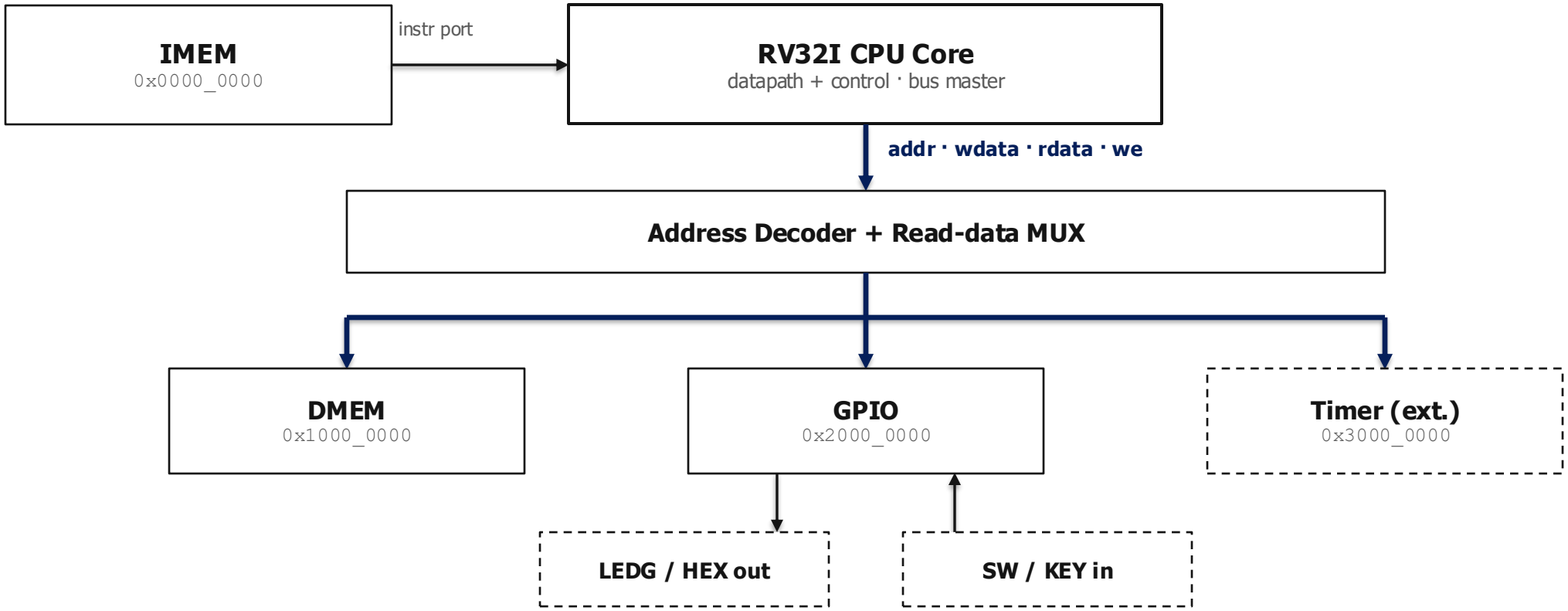
#	Step
1	Open the assembly program in RARS
2	Settings → Memory Configuration → Compact, Text at Address 0
3	Assemble
4	File → Dump Memory → select .text
5	Dump format: Hexadecimal Text
6	Save as test.hex
7	Load it in Verilog with $readmemh

Why "Text at Address 0"?

The reset PC is 0x0000_0000. If RARS places .text elsewhere, the dumped words and the branch/jump offsets no longer match the hardware.

The Complete SoC

Processor + Memory + Peripherals + Interconnect = System on Chip.



12

Lab 10 — Build Your Own RISC-V Microprocessor

Why this matters

Can you build a CPU that runs a real program? You will — module by module — and you will own the specification and the proof that it works.

Lab Objectives

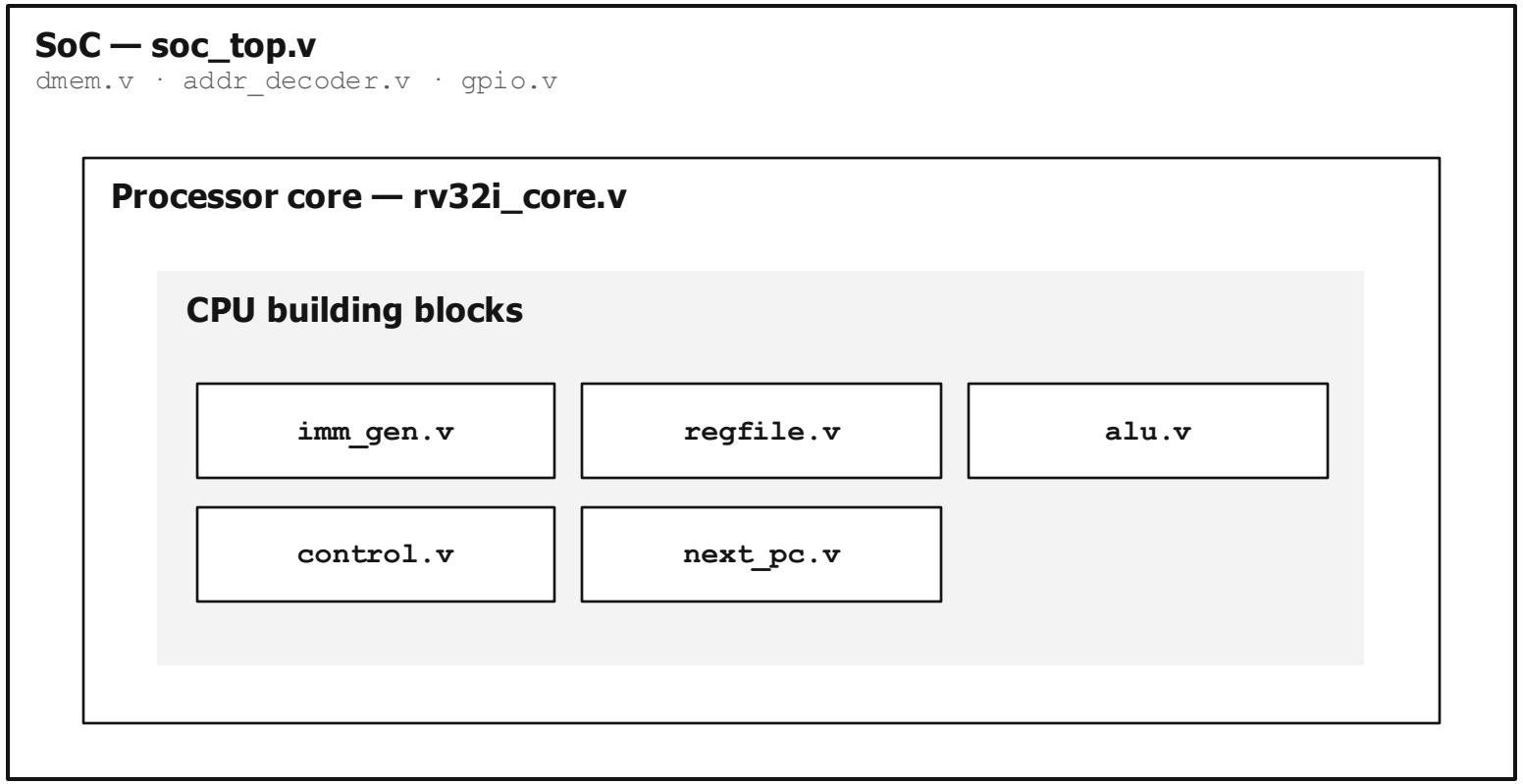
Build a working RV32I single-cycle processor in Verilog and connect it to memory and GPIO.

#	Objective
1	Build an RV32I single-cycle microprocessor in Verilog that runs a real program
2	Connect the microprocessor to DMEM and GPIO through the Address Bus
3	Verify with testbench and waveform, then run it on the DE0 board
4	Use an AI coding assistant — while you define the spec and prove correctness

Tools: VS Code + AI assistant (chat, inline, agent) · Verilog-HDL extension · TerosHDL · WaveTrace · RISC-V Support · Icarus Verilog (iverilog + vvp)

Lab Architecture: Three Layers

Build and test from the inside out: blocks pass their own testbenches, the core runs test.s, then the SoC adds bus, memory, GPIO and the board.



Layer	Proof
Blocks	tb_alu, tb_control pass
Core	tb_core runs test.s
SoC	tb_soc + DE0 demo

Skeleton + TODO: What You Get, What You Build

Modules from earlier chapters come half-done. You fill in the heart of the microprocessor.

File	Provided	Student task
imm_gen.v	Complete	Study and verify
regfile.v	Half	Write port + x0 rule
alu.v	Half	SLT (signed), SLL, SRL
control.v	TODO	Main decoder + ALU decoder
next_pc.v	TODO	PC+4, BEQ/BNE, JAL, JALR
rv32i_core.v	Skeleton	Wire the modules together
addr_decoder.v	TODO	Chip selects + read-data MUX
gpio.v	Half	LED/HEX registers + synchronizer
soc_top.v	Skeleton	Core + memory + GPIO + DE0 pins

Skeleton example: control.v

```
module control (
  input  [6:0] opcode,
  input  [2:0] funct3,
  input        funct7_5,
  output reg   reg_write,
  output reg [2:0] imm_src,
  ...
  output reg [3:0] alu_ctrl
);
// TODO: per SPEC.md §2
// default every output
// keep all port names
endmodule
```

Repository Structure

Files under tb/ come from the instructor and must not be modified — by you or by the AI. A hidden test set is used for grading.

```
lab10_rv32i_soc/
├── AGENTS.md          # rules for AI
├── SPEC.md            # you write this first
├── rtl/
│   ├── imm_gen.v      regfile.v
│   ├── alu.v          control.v
│   ├── next_pc.v      rv32i_core.v
│   ├── dmem.v         addr_decoder.v
│   └── gpio.v         soc_top.v
├── tb/                # instructor, do not edit
│   ├── tb_alu.v       tb_control.v
│   └── tb_core.v      tb_soc.v
```

```
├── sw/
│   ├── test.s / test.hex    # core test
│   └── gpio.s / gpio.hex    # board demo
├── quartus/           # DE0 project + .qsf
└── run_sim.sh         # iverilog + vvp
```

Rule

Never edit tb/. If a testbench fails, the design or the spec is wrong — not the test.

Lab Workflow — 100 Minutes

Specification comes before AI generation: Part 0 is done by hand and signed by a TA before any code is generated.

Part 0	Part A	Part B	Part C	Part D	Part E
15 min	15 min	20 min	20 min	15 min	15 min
Write SPEC.md	**Register File + ALU**	**Control + Next-PC**	**RV32I Core**	**Address Bus + GPIO**	**DE0 Demo**
Fill control, next-PC and memory-map tables by hand; TA signs	regfile.v + alu.v; pass tb_alu	control.v + next_pc.v; pass tb_control	rv32i_core.v; run test.s in tb_core	addr_decoder.v + gpio.v; pass tb_soc	Quartus: Resource Usage + Fmax; run gpio.s

Specification comes before AI generation

If your spec is wrong, the AI writes code for the wrong spec and the testbench catches it — then you fix your understanding, not just the prompt.

13

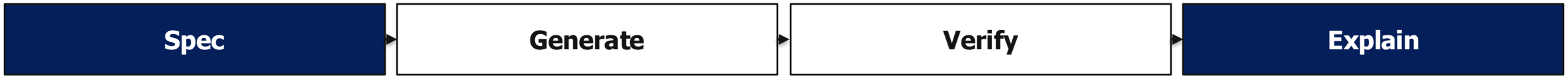
AI-Assisted Hardware Design

Why this matters

AI can write an RV32I core in seconds. If the lab only checked that code runs, you would learn nothing — so the student owns the spec and the proof.

AI Is a Tool, Not the Designer

Spec → Generate → Verify → Explain. AI-generated Verilog is a hypothesis; simulation is the evidence.



Stage	Student	AI	Evidence
Spec	Define architecture, control, next-PC and memory map — before opening AI	Not used	SPEC.md
Generate	Give precise, spec-based requirements	Generate Verilog, suggest fixes	Prompt log
Verify	Run testbenches and inspect waveforms	Analyze compiler / simulation errors, help read waveforms	PASS / FAIL
Explain	Defend every line at check-off	Not used	Check-off answers

Rules for the AI Assistant — AGENTS.md

The student owns the specification. The AI follows it.

```
# Rules for AI assistant — Lab 10
- Use Verilog-2001 only, no SystemVerilog
- Do not change module names or ports in rtl/
- Do not modify files in tb/
- Implement only from SPEC.md;
  if the spec is incomplete, ask — never guess
- Combinational: always @(*) + default outputs
- Sequential: non-blocking (<=)
- After every change: run ./run_sim.sh
  and report the result
```

Policy	Rule
Allowed	Chat, inline completion, agent mode
Required	SPEC.md done and signed by a TA before AI generation
Forbidden	Editing testbenches; asking AI for the whole core in one shot without the skeleton
Check-off	2 random questions, e.g. “point to the line that sign-extends the B-type immediate”

Bad Prompt vs Engineering Prompt

The second prompt constrains what to build, how to write it and how it will be checked — so the hardware you get back is controllable.

Weak prompt

> *"Write a RISC-V CPU for me."*

- No specification to follow
- No module interface
- No instruction list or control table
- No language or coding constraints
- No verification target

Engineering prompt

> Implement **rtl/control.v** according to the control table in **SPEC.md section 2**. Keep the **module interface** and port names exactly as in the skeleton. Use **Verilog-2001**, always @(*) with case(opcode), and give **every output a default** to avoid latches. The ALU decoder must select SUB only for R-type with funct7_5 = 1. **Do not modify tb/**; run ./run_sim.sh and report the tb_control result.

Constraint	Why it helps
SPEC.md + interface + testbench	The result can be checked, not just trusted

10 Common AI Mistakes in RV32I — and How to Catch Them

Treat each one as a verification point. For each bug ask: what symptom would appear, and which signal would you inspect?

#	AI mistake	Verification point
1	x0 can be written	addi x0, x0, 9 → x0 must still be 0
2	B-type immediate bit order wrong	BEQ with a known offset → check PCNext
3	J-type bit order / sign extension wrong	JAL forward and backward → check both targets
4	JALR target bit 0 not cleared	PCNext[0] = 0 after every JALR
5	SLT compares unsigned	slt with −1 and 5 → result must be 1
6	ADDI decoded as SUB	addi with imm −1024 must add, not subtract
7	PC used directly as IMEM index	PC 0, 4, 8 must fetch consecutive words
8	Synchronous FPGA RAM adds a cycle	LW data must appear in the same cycle
9	Decoder writes an unselected device	sw to 0x4000_0000 must change nothing
10	Switch inputs not synchronized	SW → sw_meta → sw_sync chain must exist

Debug Case 1: x0 Is Writable

It compiles and most tests pass — what is missing is the RV32I rule that x0 must always remain zero.

Suppose the AI generates this write port

```
if (we)
  regs[wa] <= wd;
```

Fix and verify

- Block writes to register 0, or force reads of x0 to return zero
- Test it deliberately:

```
addi x0, x0, 9    # expect x0 = 0
```

Debug Case 2: SLT Must Be Signed

In Verilog, reg values compare unsigned unless declared signed or cast with $signed.

```
addi x14, x0, -1    # 0xFFFFFFFF
addi x1,  x0, 5
slt  x15, x14, x1
```

Comparison	Result in x15
Signed (correct): −1 < 5	1
Unsigned (wrong): 0xFFFFFFFF > 5	0

Inspect in the waveform

- ALU operands A and B
- ALUControl = SLT?
- Signedness of the comparison
- Write-back data into x15

Debug Case 3: JALR — “x13 expected 7, got 0”

Explain the cause before changing the code.

#	Follow the path, one question at a time
1	Does x11 hold the return address 0x38?
2	Does the ALU compute x11 + immediate?
3	Is bit 0 of the target cleared?
4	Does PCNext select the JALR target?
5	Does execution return to 0x38 and run addi x13?

Good debug prompt

“tb reports FAIL x13 expected 7 got 0. Analyze dump.vcd around jalr. Do not change code yet — explain the cause first.”

14

Verification and FPGA Demo

Why this matters

A processor is correct only when the evidence says so: register values, memory contents, the PC sequence and control signals — then the board.

The Core Test Program — sw/test.s

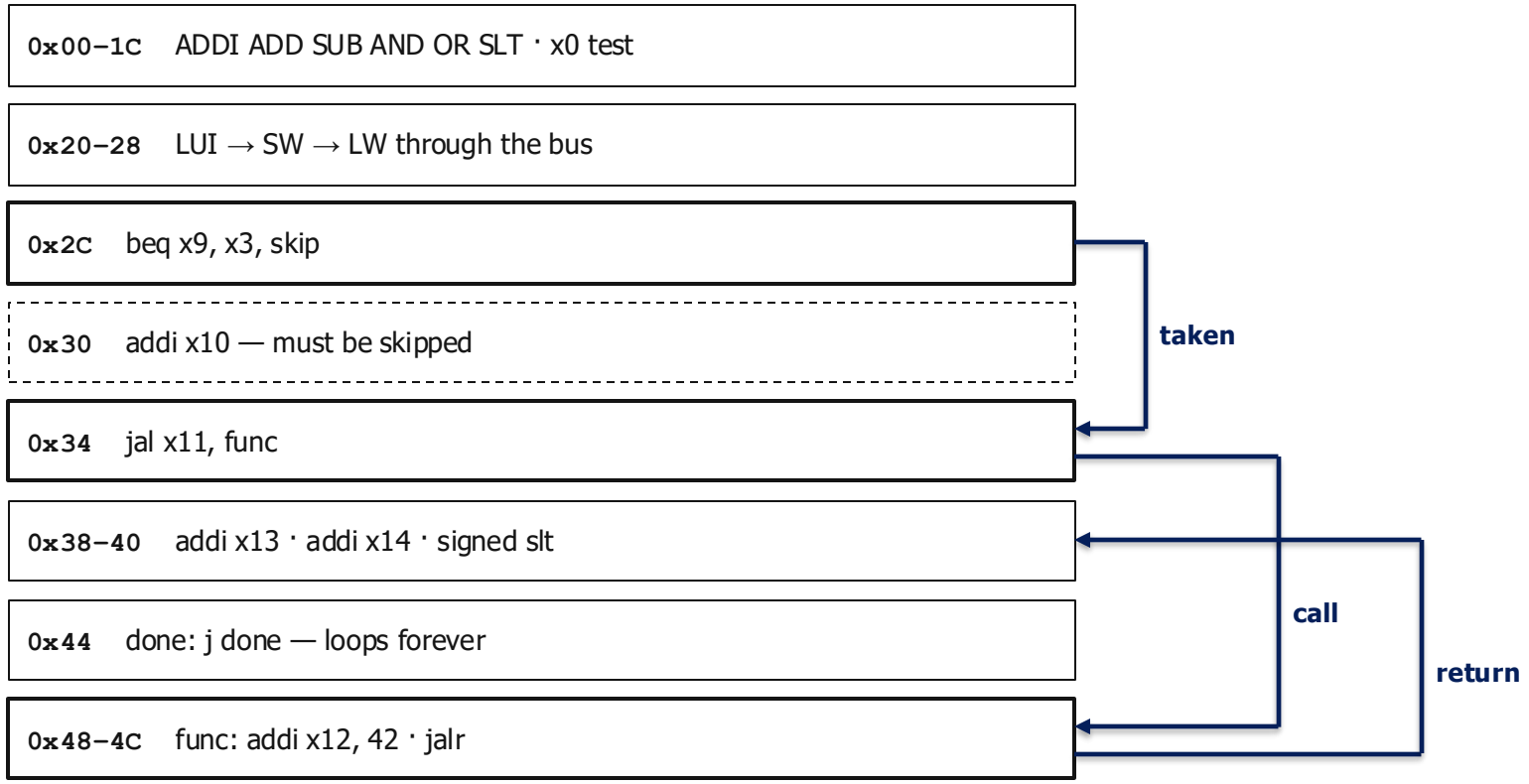
Twenty instructions exercise every instruction class, the x0 rule, memory through the bus, a branch, a call and a return.

```
0x00       addi x0,  x0, 9      # x0 unchanged
0x04       addi x1,  x0, 5      # x1 = 5
0x08       addi x2,  x0, 10     # x2 = 10
0x0C       add  x3,  x1, x2     # x3 = 15
0x10       sub  x4,  x2, x1     # x4 = 5
0x14       and  x5,  x1, x2     # x5 = 0
0x18       or   x6,  x1, x2     # x6 = 15
0x1C       slt  x7,  x1, x2     # x7 = 1
0x20       lui  x8,  0x10000    # DMEM base
0x24       sw   x3,  0(x8)      # Mem = 15
```

```
0x28       lw   x9,  0(x8)      # x9 = 15
0x2C       beq  x9,  x3, skip   # taken
0x30       addi x10, x0, 1      # skipped
0x34 skip: jal  x11, func       # x11 = 0x38
0x38       addi x13, x0, 7      # x13 = 7
0x3C       addi x14, x0, -1     # 0xFFFFFFFF
0x40       slt  x15, x14, x1    # x15 = 1
0x44 done: j    done            # stay here
0x48 func: addi x12, x0, 42     # x12 = 42
0x4C       jalr x0,  0(x11)     # back to 0x38
```

Program Flow: What the Test Checks

Arithmetic, memory through the bus, control flow (BEQ, JAL, JALR) and special cases: x0 protection, signed compare, return address, branch skip.



Checks	Instructions
Arithmetic	ADDI ADD SUB AND OR SLT
Memory	LUI → SW → LW
Control flow	BEQ, JAL, JALR
Special	x0, signed SLT, return address

Expected Register State After test.s

Each value proves one piece of hardware. A wrong value points to the module to inspect.

Reg	Expected	Tests
x0	0	x0 rule
x1	5	ADDI
x2	10	ADDI
x3	15	ADD
x4	5	SUB
x5, x6	0, 15	AND, OR
x7	1	SLT

Reg	Expected	Tests
x8	0x10000000	LUI
x9	15	SW → LW via bus
x10	0	BEQ skipped 0x30
x11	0x38	JAL return address
x12	42	Jumped into func
x13	7	JALR returned
x15	1	Signed SLT (−1 < 5)

What Should You Inspect in the Waveform?

Don't check only the final register values — follow the processor cycle by cycle.

Core signals	
PC	instruction
RD1	RD2
ImmExt	ALUResult
RegWrite	MemWrite
ResultSrc	PCNext

For BEQ

```
zero → branch_taken → PCNext
```

For JAL / JALR

```
PC+4 → write-back    (link)
target → PCNext      (jump)
```

Deliverable: waveform screenshots of BEQ and JAL.

Waveform View: BEQ → JAL → JALR

At 0x2C zero = 1 so PCNext jumps to 0x34; JAL links x11 = 0x38 and jumps to 0x48; JALR returns to 0x38 and its write to x0 has no effect.



PC	0x28	0x2C	0x34	0x48	0x4C	0x38
instr	lw x9	beq	jal x11	addi x12	jalr x0	addi x13
zero	0	1	–	0	0	0
branch_taken	0	1	0	0	0	0
Jump	0	0	1	0	1	0
PCNext	0x2C	0x34	0x48	0x4C	0x38	0x3C
RegWrite	1	0	1	1	1	1
write-back	x9=15	–	x11=0x38	x12=42	x0 (ignored)	x13=7

Address-Bus Verification — tb_soc.v

Required rule: an unmapped write must not modify any device.

Address	Selected	What to check
`0x0FFF_FFFC`	none	No write; rdata = DEAD_BEEF
`0x1000_0000`	DMEM	Only we_dmem = 1
`0x2000_000C`	GPIO	HEX register written
`0x4000_0000`	none	No device state changes

For each address ask

- Which device is selected?
- Which write enable goes high?
- What does rdata return?
- Did an unmapped write change anything?

The FPGA Program — sw/gpio.s

One program demonstrates CPU + Bus + GPIO + physical I/O: read switches, split operands, add them, display the result.

```
      lui  s0, 0x20000     # s0 = GPIO base
loop: lw   t0, 4(s0)       # read SW
      sw   t0, 0(s0)       # show on LEDG
      andi t1, t0, 0x1F    # A = SW[4:0]
      srli t2, t0, 5
      andi t2, t2, 0x1F    # B = SW[9:5]
      add  t3, t1, t2      # A + B
      sw   t3, 12(s0)      # show on HEX
      j    loop
```

Step	Instruction
Read switches	lw from 0x2000_0004
Split operands	andi / srli pick two 5-bit fields
Add them	an ordinary ALU add
Display result	sw to 0x2000_000C

Follow the Demo End to End

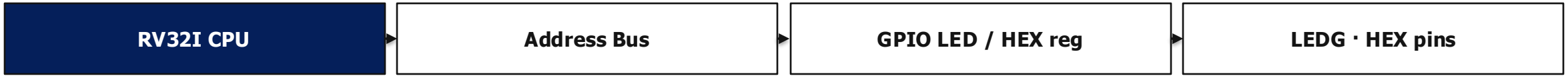
Software instructions become physical pin changes: the path in through the bus, the computation, and the path out.

IN `lw t0, 4(s0)`



CPU `andi · srli · andi → split A, B → add → A + B`

OUT `sw t0, 0(s0) and sw t3, 12(s0)`



Try it

SW = 00011_00101 → A = 5, B = 3 → HEX shows 8.

15

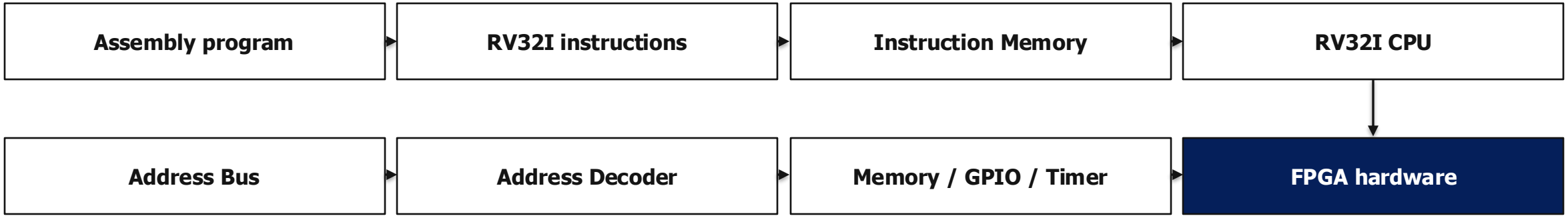
Wrap-up

Why this matters

A processor becomes a System on Chip when computation, memory, communication and peripherals are integrated into one addressable system.

The Complete Journey

ALU + Registers → Datapath → + Control → RV32I Microprocessor → + Memory → + Address Bus and Decoder → Memory-Mapped I/O → System on Chip



Definition

A processor becomes a System on Chip when computation, memory, communication and peripherals are integrated into one addressable system.

For the CPU, everything is memory.

The Key Idea Revisited

The same lw / sw mechanism reaches Data RAM, the LED register, the switch register, a timer, a UART — or a future accelerator.

#	To add a new peripheral
1	Assign a new address in the memory map
2	Add its chip select in the address decoder
3	Connect its write enable = we & sel
4	Add its data to the read-data MUX

What does not change

The CPU architecture never needs a new instruction per peripheral.

Connection to the SoC Mini-Project

This SoC is the foundation of your mini-project. Hardware becomes programmable through the memory map.

PWM	**UART**	**Timer**
Sensor interface	**MAC unit**	**AI accelerator**

```
0x4000_0000 → Custom Accelerator
```

Software then controls it with ordinary loads and stores.

AUDIENCE — ANSWER FIRST

Review Challenge — Answer Without Looking Back

Discuss in pairs first. Answers are covered in the lab check-off.

#	Question
1	Why must a single-cycle CPU keep IMEM and DMEM separate?
2	Why is x0 useful? Which instructions (mv, nop, j) rely on it?
3	Which instruction sets the single-cycle clock period, and why?
4	Write every control signal for sw x5, 8(x2).
5	At 0x0000_0020, where does a taken beq x1, x2, -8 go?
6	Full vs partial decoding — give an aliasing example from our memory map.
7	sw to 0x4000_0000 has no device. What should the hardware do, and what should lw return?
8	Why must switch inputs pass a 2-flip-flop synchronizer?
9	Write short assembly that blinks LEDG[0] using the button at 0x2000_0008.
10	AI wrote imm_gen.v. Which test cases prove the B-type immediate is right?

Final Takeaways

Six ideas to keep from this chapter.

Processor

The datapath executes operations; control selects the correct path.

RV32I

The ISA is the contract between software and hardware.

Single-cycle

Simple and CPI = 1, but limited by the longest path (LW).

SoC

The CPU talks to memory and peripherals through addresses on a bus.

Memory-mapped I/O

LW and SW can read and drive physical devices.

Engineering with AI

Spec → Generate → Verify → Explain.

Lab Mission: Build It. Simulate It. Explain It. Run It.

More than half the points are spec, explanation and verification — the parts AI cannot do for you.

#	Deliverables
1	SPEC.md signed by a TA
2	All Verilog in rtl/
3	Every testbench PASS (screenshots)
4	BEQ and JAL waveforms
5	Quartus Resource Usage + Fmax
6	DE0 demo video or check-off
7	AI Usage Log — incl. one case where AI was wrong

Grading	Points
SPEC.md (control, next-PC, memory map)	20
Provided testbenches pass	20
Hidden tests pass	15
DE0 demo	15
Check-off: explain your code	20
AI Usage Log	10
Bonus: Timer or LB/SB with wstrb	+10

From One Instruction to a Physical LED

That is the bridge from software to digital hardware.

```
sw t0, 0(s0)
```

For the CPU, everything is memory.

